学 校 代 码 10459
学号或申请号 202122172013296
密 级 公开

郑州大学

专业硕士学位论文

基于MLIR的深度学习模型编译优化技术研究

作 者 姓 名：桂中华
导 师 姓 名：徐金龙
专业学位名称：计算机技术
培 养 院 系：计算机与人工智能学院
完 成 时 间：2024年5月

中国知网 https://www.cnki.net

A thesis submitted to

Zhengzhou University

for the degree of Master

Research on MLIR based deep learning models compilation optimization technology

By: Zhonghua Gui

Supervisor: Jinlong Xu

Computer Technology

School of Computer and Artificial Intelligence

May, 2024

中国知网 https://www.cnki.net

摘 要

MLIR(Multi-level Intermediate Representation)编译器在人工智能领域具有广泛的应用，利用其进行深度学习模型优化以提升执行效率是当前重要的研究方向。低精度模型量化是深度学习领域的重要加速技术之一，然而当前 MLIR 编译器并不支持 FP8 数据类型的量化，不能对深度学习模型进行 FP8 量化评估，同时基于 MLIR 中间表示的推理引擎内存分配策略存在冗余，未能有效利用内存资源，针对更大的模型推理存在潜在的内存瓶颈。此外 MLIR 编译器的算子融合优化考虑不够全面，限制了 MLIR 编译器的优化能力，影响了模型中算子的执行效率。为了解决以上问题，本文基于 MLIR 编译器增加 FP8 数据类型量化策略，改进内存分配策略以及针对算子融合进行了深度优化。

本文主要工作和创新点如下：

（1）基于 MLIR 的 FP8 类型量化支持。本文首先梳理了 MLIR 编译器中模型量化的主要流程，研究了 MLIR 编译器对深度学习模型的量化类型支持及精度损失，根据分析结果在编译器中添加 FP8 量化类型支持，实现了浮点精度转换算法，重构了 MLIR 量化模块中的类型验证和解析模块，在 FP8E4M3 和 FP8E5M2 两种数制下进行 CPU 推理实验，对多种深度学习模型的推理精度损失进行模拟。实验表明，利用 FP8 类型进行量化，在 Yolov5s 和 Resnet50 模型的测试中精度优于 INT8，同时部署更为简便。

（2）基于 MLIR 的推理引擎内存优化。通过对基于 MLIR 中间表示的推理引擎的执行流程以及内存分配策略的梳理和研究发现，当前推理引擎基于算子库分发，内存分配策略以推理速度为主，采用静态内存分配方案，存在分配冗余。本文基于 MLIR 中间表示提出了一种基于静态分配的内存复用优化算法，结合模型特征对内存分配过程进行深度优化。在 Resnet50 和 Yolov5s 等模型的测试中，实验表明，利用该优化策略可以减少内存 4%~20%。

（3）基于 MLIR 的算子融合支持与优化。针对深度学习模型中的 SwiGLU 算子的计算模式分析发现，该算子在 MLIR 中间表示中同时存在垂直位置和水平位置可融合的子算子，当前 MLIR 算子融合策略仅支持垂直位置算子融合而忽略了水平位置的算子，本文提出了基于 MLIR 中间表示的垂直加水平融合优化策略。为了解决算子库对推理性能的限制，本文进一步改善了基于 MLIR 的

中国知网 https://www.cnki.net

自动算子生成策略，针对融合后的算子存在的访冗余进行优化。实验表明，该融合策略可以将算子的推理时间提升高达 9%。

本文工作基于 CPU 和 GPU 两种硬件后端，分别在基于 MLIR 编译器的 TPU-MLIR 编译框架和 IREE 编译器上予以实现。通过基准测试集进行测试与编译器当前版本进行对比，验证了优化工作的正确性以及有效性，实验表明本文提出的 FP8 量化方法对模型量化部署有一定参考意义，内存优化和水平算子融合优化有效的提升了模型在硬件上的内存性能和执行效率。

关键词：MLIR 编译器；FP8 量化；内存优化；算子融合；代码生成；深度学习

中国知网 https://www.cnki.net

Abstract

The Multi-level Intermediate Representation compiler has a wide range of applications in the field of artificial intelligence, and using it for deep learning model optimization to improve execution efficiency is currently an important research field. Low precision model quantization optimization is one of the important acceleration technologies in the field of deep learning. However, the current MLIR compiler does not support quantization of FP8 data types and cannot perform FP8 quantization evaluation on deep learning models. At the same time, the inference engine memory allocation strategy based on MLIR intermediate representation has redundancy and fails to effectively utilize memory resources. There is a potential memory bottleneck for inference on larger models. In addition, the operator fusion optimization of MLIR compiler is not comprehensive enough, which limits the optimization ability of MLIR compiler and affects the execution efficiency of operators in the models. In order to solve the above problems, this article adds FP8 data type quantization strategies based on the MLIR compiler, improves memory allocation strategies, and conducts deep optimization for operator fusion.

The main work and innovative points of this article are as follows:

Firstly, FP8 type quantization support based on MLIR. This article first outlines the main process of model quantization in the MLIR compiler, studies the quantization type support and accuracy loss of deep learning models in the MLIR compiler, adds FP8 quantization type support to the compiler based on the analysis results, implements floating-point precision conversion algorithm, and reconstructs the type verification and parsing module in the MLIR quantization module. CPU inference experiments are conducted at two types, FP8E4M3 and FP8E5M2, to simulate the inference accuracy loss of various deep learning models. The experiment shows that using FP8 type for quantization has better accuracy than INT8 in the testing of Yolov5s and Resnet50 models, and is easier to deploy.

Secondly, Memory optimization of inference engine based on MLIR. Through

中国知网 https://www.cnki.net

the analysis and research of the execution process and memory allocation strategy of the inference engine based on MLIR intermediate representation, it is found that the current inference engine is based on operator library distribution, and the memory allocation strategy is based on inference speed and adopts a static memory allocation scheme, which leads to allocation redundancy. This article proposes a static allocation based memory reuse optimization algorithm based on MLIR IR, which combines model features to deeply optimize the memory allocation process. In the testing of models such as Resnet50 and Yolov5s, experiments have shown that using this optimization strategy can reduce memory by 4% to 20%.

Thirdly, Support and optimization of operator fusion based on MLIR. Analysis of the computational patterns of the SwiGLU operator in deep learning models reveals that there are sub operators in the MLIR IR that can fuse both vertical and horizontal positions. The current MLIR operator fusion strategy only supports vertical position operator fusion and ignores horizontal position operators. This paper proposes a vertical and horizontal fusion optimization strategy based on the MLIR intermediate representation. In order to address the performance limitations of the operator library, this paper further improves the automatic operator generation strategy based on MLIR and optimizes the access redundancy of the fused operators. Experiments have shown that this fusion strategy can increase the inference time of operators by up to 9%.

The article is based on two hardware backends, CPU and GPU, and is implemented on the TPU-MLIR compilation framework based on the MLIR compiler and the IREE compiler, respectively. By comparing the benchmark test set with the current version of the compiler, the correctness and effectiveness of the optimization work were verified. The experiments showed that the FP8 quantization method proposed in this paper has certain reference significance for model quantization deployment. Memory optimization and horizontal operator fusion optimization effectively improve the model's hardware memory performance and operator execution efficiency.

Key words: MLIR; FP8 quantification; Memory optimization; Operator fusion; Code generation; Deep learning;

中国知网 https://www.cnki.net

英文专业缩写明细表

英文缩写	英文全称	中文名称
MLIR	Multi-level Intermediate Representation	多级中间表示
LLVM	Low Level Virtual Machine	底层虚拟机
XLA	Accelerated Linear Algebra	线性代数优化编译器
TVM	Tensor Virtual Machine	张量虚拟机
IREE	Intermediate Representation Execution Environment	中间表示执行环境
PTQ	Post-Training Quantization	训练后量化
QAT	Quantization Aware Training	量化感知训练
IR	Intermediate Representation	中间表示
SSA	Static Single Assignment	静态单赋值
CUDA	Compute Unified Device Architecture	统一计算设备架构/模型（基于 NVIDIA）
SM	Stream Multiprocessor	多流处理器
SIMT	Single Instruction Multiple Thread	单指令多线程
ONNX	open neural network exchange	开放神经网络交换格式
D-U	Define Use Chain	定义使用链

中国知网 https://www.cnki.net

目 录

1 绪论.....1
1.1 课题研究背景及意义.....1
1.2 国内外研究现状.....3
1.3 研究内容.....5
1.4 组织结构.....5
2 相关理论基础.....8
2.1 MLIR 编译器.....8
2.2 模型量化.....10
2.2.1 模型量化简介.....10
2.2.2 FP8E4M3 和 FP8E5M2.....13
2.3 算子融合优化与硬件架构.....15
2.3.1 算子融合优化技术.....15
2.3.2 CPU+GPU 异构平台.....16
2.4 本章总结.....18
3 基于 MLIR 的 FP8 量化策略支持.....19
3.1 研究动机.....19
3.2 实现方法.....19
3.2.1 值转换规则构建.....20
3.2.2 量化类型与属性构建.....22
3.2.3 缩放因子选取.....24
3.3 实验结果与分析.....25
3.3.1 实验环境.....25
3.3.2 测试结果与分析.....28
3.4 本章总结.....30
4 基于 MLIR 的推理引擎内存优化.....31
4.1 研究动机.....31
4.2 实现方法.....32
4.2.1 内存分配策略分析.....33
4.2.2 内存复用策略支持.....34
4.3 实验结果与分析.....36
4.4 本章总结.....38
5 基于 MLIR 的水平算子融合优化.....39
5.1 研究动机.....39
5.1.1 问题研究.....39

中国知网 https://www.cnki.net

5.1.2 相关工作.....41
5.2 实现方法.....41
5.2.1 SwiGLU 算子计算模式分析.....41
5.2.2 水平融合策略支持.....43
5.3 实验结果与分析.....48
5.4 本章总结.....50
6 总结与展望.....51
6.1 工作总结.....51
6.2 未来展望.....52
参考文献.....53

中国知网 https://www.cnki.net

图和附表清单

图 1.1 篇章结构 6
图 2.1 Dialect 结构 8
图 2.2 可复用 Dialect 9
图 2.3 FP32、FP16 与 BF16 格式对比 12
图 2.4 FP32 与 FP8E4M3、E5M2 格式对比 13
图 2.5 主机端调用核函数运行过程 16
图 2.6 GPU 整体硬件架构 16
图 2.7 CUDA 编程模型 17
图 3.1 FP32 到 FP8 的映射过程 21
图 3.2 FP8 量化类型构建 23
图 3.3 量化属性构建 23
图 3.4 基于 MLIR 的 TPU-MLIR 编译框架 27
图 3.5 COCO2017 下 Yolov5s mAP 测试 IOU=0.5 28
图 3.6 COCO2014 下 Yolov5s mAP 测试 IOU=0.5 29
图 3.7 Resnet50 模型精度对比 29
图 4.1 SILU 算子内存复用支持 35
图 4.2 内存占用峰值对比 37
图 5.1 语言模型中的 SwiGLU 算子 40
图 5.2 MLIR 中间表示描述的 SwiGLU 算子 42
图 5.3 SwiGLU 计算模式 43
图 5.4 MLIR 算子融合规则 44
图 5.5 MLIR 自动算子融合后的 SwiGLU 算子 45
图 5.6 水平加垂直融合后的 SwiGLU 算子 47
图 5.7 存在访存冗余的 SwiGLU 算子 48
图 5.8 访存冗余消除后的 SwiGLU 算子 48
图 5.9 基于 MLIR 的 IREE 编译器架构 49
图 5.10 不同输入下的优化前后推理时间对比 50
表 2.1 FP8E4M3 二进制编码规则 14
表 2.2 FP8E5M2 二进制编码规则 14
表 3.1 测试集 27

中国知网 https://www.cnki.net

1 绪论

随着人工智能领域目标检测模型和语言大模型的迅速发展，深度学习模型越来越多样，模型也变的越来越庞大。借助编译器开展模型优化以提升模型性能受到广泛关注，其中 MLIR 编译器因其具有良好的可拓展性与 LLVM(Low Level Virtual Machine)生态兼容等特点成为当前研究的热点。深度学习模型优化技术中，模型量化和算子融合技术分别在模型轻量化和硬件资源利用方面展开优化，结合 MLIR 编译器实现模型的自动优化以高效部署到硬件。

本章首先阐述研究的背景和意义，然后对模型量化技术和算子融合技术的国内外研究现状展开介绍，接着提出了本文的研究内容和创新点，最后阐述本文的文章结构。

1.1 课题研究背景及意义

当前人工智能算法的开发和部署往往使用 TensorFlow[1]与 PyTorch[2]两大平台，随着 Pytorch2.0 中编译技术的加入和 TensorFlow 后端编译器作为独立项目 OpenXLA[3]开展,当前深度学习模型的优化逐步由框架转向编译器。除此之外由 Tianqi Chen 等人发布的 TVM(Tensor Virtual Machine)[4]经过近几年的发展已经成为当前最为主流的深度学习编译器项目。MLIR[5]是由 LLVM[6]作者 Chris Lattner 等人于 2019 年开源的编译基础设施项目。作为 LLVM 的子项目，MLIR 是一种构建可重用、可扩展编译器基础结构的方法，旨在解决软件碎片化，改进异构硬件编译过程，降低构建领域特定编译器的成本。MLIR 的核心理念是多层抽象，逐步递降，部分递降以开展更为全面的优化，编译器内部也提供了大量的抽象层以供搭建特定编译器进行复用。目前基于 MLIR 基础设施构建的项目越来越多，其中国内算能开源的 TPU-MLIR[7]和国外 Google 开源的 IREE(Intermediate Representation Execution Environment)编译器[8]在时间节点上出现最早，编译解决思路具有一定代表性。

基于 MLIR 编译器开展深度学习模型优化主要包括算子间优化、算子生成、量化、动态张量形状和稀疏问题的优化，其中模型量化技术主要是面对快速增长的模型规模和硬件算力、存储增长间的不平衡，结合深度学习网络模型的鲁

中国知网 https://www.cnki.net

棒性以低比特数据存储与运算，通过压缩模型空间以更好的进行模型部署，现有的模型量化策略主要基于 FP16、BF16、INT8 等类型实现。

深度学习模型中算子融合技术是 MLIR 中的主流优化技术，根据深度学习模型中算子间关系分为水平融合和垂直融合，其中以垂直融合最为常见，主要通过减少连续算子间冗余的数据搬运，内核启动次数以提升模型性能，水平融合主要面向模型中位于水平轴上的算子，通过降低单独算子的内核启动开销，改善 GPU 负载以提升程序性能。

目前，MLIR 编译器在开展模型量化以及模型的算子融合优化已经取得了不错的效果。然而随着深度学习领域的发展，新的低位宽数据类型被提出，同时算子的种类越来越多，计算模式越来越复杂，尽管 MLIR 已经支持了其中的一部分，但仍然存在一些问题需要解决。

（1）当前低位宽模型量化技术中 8bit 数据类型在降低推理内存占用与部署开销方面最为显著，但 MLIR 仅支持 INT8 量化。对于新提出的数据类型，MLIR 并不支持，也无法通过对模型进行新类型量化精度评估以选择最佳的量化部署方案。

（2）基于 MLIR 中间表示的推理引擎在对模型进行推理时，采用了一次性分配的静态内存分配方案，根据模型遍历为每个算子都分配对应的输入和输出空间，没能结合模型中算子的特点，未能有效的利用内存，对于更大的一些模型可能存在内存上限的问题。

（3）算子融合技术是 MLIR 中提升模型性能的主要方式之一，但其融合规则对于计算模式比较复杂的算子会忽略了潜在的融合机会，同时针对融合后的算子，MLIR 自动代码生成策略存在一定优化空间，限制了 MLIR 编译器的优化性能。

本文借助 MLIR 编译器对新数据类型进行量化支持，通过在编译器内构建相关模块支持模型进行新类型的量化以及推理，在推理过程中针对推理引擎存在的内存分配问题，结合模型特征对内存分配过程进行优化。同时对于 MLIR 的算子融合策略展开了深入研究，对模型中的算子进行更全面的融合。基于 MLIR 开展以上深度学习模型相关优化 ，为人工智能算法研发和部署提供支撑，在人工智能算法快速发展和硬件受限的今天，具有一定的研究价值。

中国知网 https://www.cnki.net

1.2 国内外研究现状

MLIR 是一种编译基础设施，具有良好的可拓展性，为快速构建领域特定编译器提供支持，其中国内首个基于 MLIR 的开源编译器项目是由算能公司开发的 TPU-MLIR，通过 MLIR 框架构建编译流将深度学习模型快速部署到其 TPU 平台；阿里 PAI 团队基于 MLIR 编译器开发的开源编译器 BladeDISC[9]为深度学习模型中出现的动态形状问题提供编译优化解决方案；中科院计算的张洪滨等人所基于 MLIR 编译器开发的 Buddy-MLIR[10]为 RISC-V 芯片端深度学习模型部署提供编译优化解决方案，除此之外国内诸多厂商如遂原科技、寒武纪、字节跳动、地平线和智源研究院等[11]都有基于 MLIR 编译器开发自家深度学习编译器。国外比较具有代表性的有 Google 基于 MLIR 开发的开源编译器 IREE[12]，旨在提供端到端的多后端全流程编译解决方案，OpenAI 基于 MLIR 开发的开源编译器 Triton[13]旨在提供一种快速搭建 GPU 高性能算子的解决方案，除此之外还有 Intel 基于 MLIR 开发 TPP-MLIR 编译器和 nGraph 编译器[14]，知名 RISC-V 公司 SiFive 基于 MLIR 打造编译解决方案等[15]。

在模型优化技术中，模型量化技术针对 FP16 类型进行模型推理已经较为成熟，早在 2008 年 IEEE 浮点运算标准就引入了 FP16 作为存储格式，当前部分硬件针对深度学习模型默认支持 FP16 类型推理。INT8 量化[16]是 Google 在 TensorFlow Lite 框架中使用的量化方案。将 FP32 类型数据转换为 INT8 类型，相当于对数据进行 8bit 再编码，这一方法目前被使用广泛使用，作为低精度量化的主流方式。Sun 等人在 2019 年提出了 HBF8 格式[17]，其中包含 1 位符号位、4 位阶码位和 3 位小数位，阶码的偏移值设置为 11，通过实验表明该格式量化在 AlexNet、DenseNet121 和 MobileNetV2 等模型中获得接近 FP32 的精度。英伟达联合 AMD、ARM 等公司于 2022 年发布了 FP8 格式标准[18]，其中包含 FP8E4M3 和 FP8E5M2 两种格式，并给出主流深度学习模型的实验结果，但当前 MLIR 平台仍缺乏对 FP8 量化格式的支持，同时缺乏对更多主流模型的精度测试。

算子融合作为深度学习编译器中模型推理加速的一种关键技术，主要通过减少访存和内核启动次数改善模型性能，是许多框架和深度编译器提升模型性能的主要途径之一。TensorFlow 框架对深度学习模型进行子图切分后借助 XLA(Accelerated Linear Algebra)编译器[19]对进行编译优化，XLA 对子图进行算子融合优化，子图优化相对独立但不能够跨子图优化，缺少融合机会。同时 XLA

中国知网 https://www.cnki.net

使用专家定义的规则以融合模型中的算子。对于一些复杂的算子或新的模型算子，框架仍然需要基于专家经验开发算子融合规则，缺乏一定的灵活性。Ma L 等人[20]提出了自动编译框架 Rammer，主要用于优化在并行计算能力较强的 GPU 和 ASIC 硬件加速器上执行的深度学习模型，在编译时对深度学习模型转换为基于 rOperators 的数据流图和更细粒度的 rTask，通过算子融合等优化技术对计算负载进行编排，该方案打破了计算图与算子间的隔阂，对算子之间和算子内部的协同调度整体地利用并行性，使硬件利用率最大化。Niu W 等人[21]提出了算子融合框架 DNNFusion，并未使用基于固定模型匹配的算子融合策略，该框架将深度学习模型中的算子进行了类型分类，根据算子的类别进行融合，使得算子融合策略更加通用。阿里巴巴团队为了进一步提升深度学习模型中算子的融合效果，在 PAI 平台上引入了 AStitch 技术[22]。该技术是一种大粒度计算融合的编译优化手段，系统化地将常见的算子融合模式分为四类，通过计算图的依赖关系特性、GPU 多层次存储架构上的数据局部性、以及不同数据尺寸之下的线程并发性等三个方面的联合考虑，自动化地为大粒度的复杂访存密集算子子图生成高效的 GPU 侧代码。

TianqiChen 等人提出了 TVM 编译器,基于 Relay 层开展计算图融合，通过在 Relay 层对算子进行分类，基于类别进行算子融合，通过使用基于深度学习的 Ansor 技术对融合后的算子进行代码生成和优化，但针对一些新的算子关系和算子类型以及计算复杂的算子支持仍不足，同时其基于深度学习的算子生成技术耗时较长。微软亚洲研究院基于 Rammer 提出了 ROLLER[23]利用了新的 rTile 抽象 ,将少量 shape 与多种硬件特征进行对齐，基于递归结构的算法在短时间内生成高效的内核，大大减少融合后算子的生成时间，同时性能可与 GPU 等硬件库相媲美。Jia Z 等提出了 TASO[24]，通过自动生成图替换策略对深度学习模型的计算图进行优化。金雪峰、赵捷等人[25]在华为 Mindspore 框架的中提出一种图层和算子层融合的表达方式，打破了图层和算子层的边界，探索了更多的融合机会。同时后端基于多面体模型进行算子生成[26]，利用空间几何的仿射变换来实现循环级别的融合，该方案针对计算密集型线性代数算子具有较好的效果，但对于一些复杂的难以仿射的算子融合不足。在 MLIR 编译器中主要在 Lianlg Dialect 的 Transform 模块开展算子融合优化[27]，通过对算子分类，将符合规则的算子融合到一个 Lianlg Operation 的负载域中进行任务分发。总之，尽管目前国外算子融合技术已经得到广泛的研究和应用，面向深度学习模型的快速发展，

中国知网 https://www.cnki.net

MLIR 编译器中的算子融合技术在针对特定算子时缺乏融合机会，不够全面，在 MLIR 编译器中开展算子间优化提升模型性能仍有一定的研究空间。

1.3 研究内容

MLIR 编译器作为编译器基础设施提供了一套构建编译大厦的脚手架，本文针对基于 MLIR 的编译器展开研究，针对 MLIR 编译器中的量化模块进行 FP8 类型支持以及针对 CPU 的推理引擎展开优化。同时，对于编译器中的算子融合策略进行改进，对融合策略进行扩展以生成高效的 GPU 内核，全文一共分为六章，其篇章结构如图 1.1 所示：

（1）基于 MLIR 的 FP8 类型量化支持。本文首先梳理了 MLIR 编译器中模型量化的主要流程，研究了 MLIR 编译器中对于深度学习模型的量化类型支持及精度损失情况，通过在编译器内添加 FP8 量化类型支持和不同精度间类型转换算法，分别在 FP8E4M3 和 FP8E5M2 两种数值下进行了模型推理实验，对模型的精度损失进行模拟。对深度学习模型在 FP8 类型的量化下精度损失情况和 FP8 量化策略的部署难度进行系统评估。

（2）基于 MLIR 的推理引擎内存优化。在验证 FP8 类型量化精度损失的过程中，使用了基于算子库的推理引擎，对该引擎的执行流程以及内存分配策略进行了梳理和研究，发现当前的内存分配策略以推理速度为主要考虑因素，采用静态内存分配策略，存在一定的压缩空间，因此基于 MLIR 中间表示进行推理引擎的内存复用优化，研究该优化策略对深度学习模型的内存占用的影响。

（3）基于 MLIR 编译器下的水平算子融合支持与优化。首先梳理了 MLIR 编译器中的算子融合策略，接着对深度学习模型中出现的 SwiGLU 算子的计算模式进行分析，发现当前融合策略针对 SwiGLU 算子存在优化空间，针对该算子展开深度融合优化。由于算子融合优化的性能受算子库限制，本节进一步研究了基于 MLIR 的自动算子生成策略，针对融合后的算子存在的访冗余进行优化，研究算子融合优化策略对 SwiGLU 算子推理延迟的影响。

1.4 组织结构

本文基于 MLIR 编译器开展深度学习模型研究，主要在 CPU 端和 GPU 端进行测试，以在 MLIR 平台支持 FP8 数据量化模拟为基础，对基于 MLIR 中间表

中国知网 https://www.cnki.net

示的推理引擎内存分配开展优化工作，该部分实验在 CPU 侧开展。本文通过对 MLIR 编译器的算子融合策略、SwiGLU 算子的计算模式以及 MLIR 编译器的算子生成策略进行研究，提出垂直加水平的算子融合策略并结合 MLIR 编译器的算子生成技术，对融合后的算子在 GPU 侧存在的访存冗余进行优化。全文共分为六章，其文章篇章结构如图 1.1 所示：

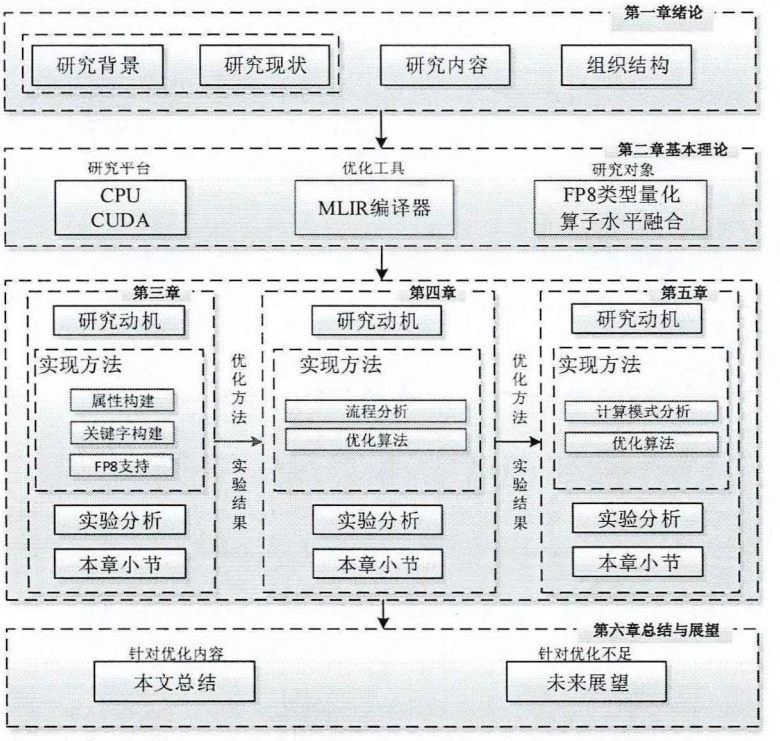


图 1.1 篇章结构

本文在第一章和第二章介绍了研究的平台、背景、工具和对象，为第三、四、五章节提供了研究基础。第三、四、五章节为本文的研究内容，在每一章节中提出优化方法的研究动机、实现方法、实验分析和总结。其中，第四章的研究内容以第三章研究实验过程中发现的问题为研究对象，第五章以第三、四章中实验中性能受限问题为基础开展研究。最后，本文针对优化内容做出了总结，针对优化不足对未来做出展望。各个章节详细内容如下：

第一章节主要介绍了本文的研究背景、国内外研究现状以及本文的研究内容并列举了创新点。

第二章节介绍了本文的研究平台、优化工具以及研究对象。首先详细描述了 MLIR 编译器设计，针对编译器的背景、特点及编译流程进行介绍。然后对

中国知网 https://www.cnki.net

深度学习模型优化技术中的模型量化和算子融合优化进行介绍，并说明使用的硬件信息。

第三章首先分析了为 MLIR 编译器提供 FP8 数据类型量化的研究动机，接着介绍了 FP32 类型和 FP8 类型的互相转换算法，然后根据此算法提出了一种针对 FP8 类型的量化方案，最后描述了本文实验平台的相关参数，利用实验验证了量化方案的正确性，并对实验结果进行了详细的分析和对比。

第四章基于第三章实验环节使用的推理引擎存在的问题开展研究。首先分析了基于 MLIR 中间表示的 CPU 推理引擎的执行流程，接着针对其中的内存分配问题进行研究，考虑到基于算子库类型的推理引擎难以修改算子库存在性能受限的情况，设计一种基于定义使用链的原地内存复用策略。最后通过在 CPU 上实验，验证了该策略的正确性和有效性。

第五章节基于第三、四章中使用算子库进行实验这种性能受限的场景，进行深度学习模型的算子融合及算子生成技术研究，并提出一种算子深度融合策略。本章首先分析了深度学习模型中 SwiGLU 算子的计算模式，接着根据该算子在编译器中间代码的表示结构，构造 Pass 优化遍以实现一种垂直加水平融合的优化方案，最后利用编译器的算子生成策略进行优化和硬件部署，通过实验验证了该优化方法的正确性和有效性，并对实验结果进行了分析。

第六章节对本文的研究内容做出了总结并叙述了研究内容和方法的局限性，最后面向本文对当前基于 MLIR 的深度学习编译器研究做出展望。

中国知网 https://www.cnki.net

2 相关理论基础

本章首先介绍 MLIR 编译器，对编译器的组织结构与内置的模块进行概述，接着介绍基于 MLIR 编译器开展的模型量化和算子融合技术，最后介绍了实验使用的硬件平台相关的内容。

2.1 MLIR 编译器

MLIR 是一种用于构架编译器的基础设施，作为 LLVM 的子项目，其提供了一些通用的、可重用的中间表示模块和工具链，包括一组库、工具和通用的 IR 中间表示以及用于完成编译对象的转换、分析或优化等功能的 Pass 优化遍。

MLIR 中间表示的核心概念主要包括 Dialect、Operation、Attribute、Type、Interface、Constraint、Trait、Region、Block 等。

（1）Dialect 在特定的命名空间下为抽象提供了分组机制，能够将所有的抽象放在同一个命名空间中，并分别为每种抽象定义对应的 MLIR 表达式并绑定对应的 Operation，生成 MLIR 中间表示。

（2）Operation 是 Dialect 中具有特定语意的抽象和计算的核心单元，能够表示指令、函数、模块等。

（3）Attribute 是属性信息，Type 是类型信息，Interface 是标准化的接口。

（4）Constraint 和 Trait 是 Operation 的限制信息和特征信息。

（5）Region 和 Block 是 Operation 的作用域，支持嵌套使用。

Dialect 结构如图所示，其中，Operation 主要包括 Attribute、Type、Interface、Constraint、Trait、Region 和 Block，Dialect 主要包括 Operation、Attribute、Type 和 Interface。

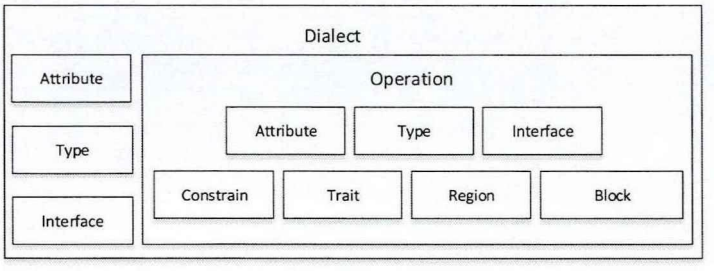


图 2.1 Dialect 结构

中国知网 https://www.cnki.net

当前 MLIR 编译系统中存在一些可供复用的 Dialect[28]在不同的抽象层次进行计算和结构描述以进行特定优化，常用的 Dialect 如 Linalg、Vector、GPU、Arith、SCF 等图 2.2 所示。

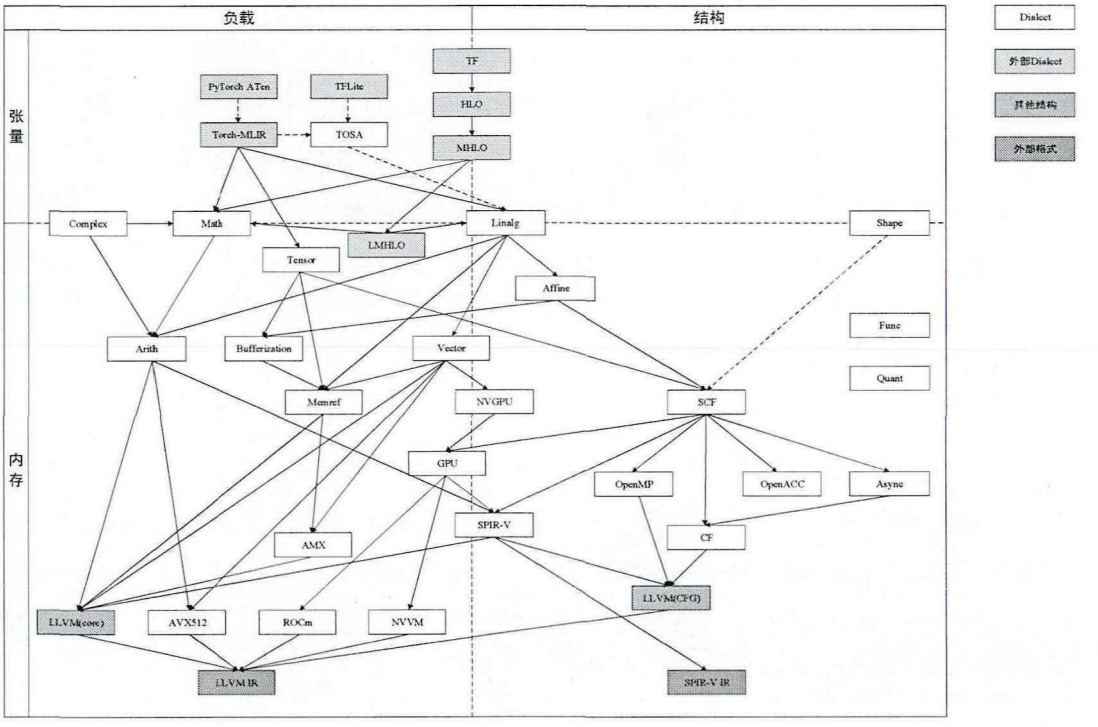


图 2.2 可复用 Dialect

• **Torch、TOSA、StableHLO(MHLO)**等 Dialect 处于 MLIR 编译系统的边界，用于接入深度学习框架中 Python 搭建的或 ONNX 描述的深度学习模型，用于描述模型中由算子组成的 high-level 计算图，类似 Pytorch Aten、TVM Relay 等只描述算子的类型，输入输出，属性以及算子间的依赖关系，由于抽象层次较高，并不描述算子的计算细节和硬件相关信息。

• **Linalg Dialect** 处于 MLIR 编译系统的中层用于递降以将高层描述的算子进一步细分，Linalg 主要用于描述线性代数相关的结构，本质是完美嵌套循环，包含索引指定循环变量如何访问操作数和结果，执行模式如并行、规约模式为进一步递降提供参考，计算负载域用于描述参与的计算。Linalg 还可以进行深度学习编译器中常见的算子融合和分块优化。

•**Affine Dialect** 借鉴了多面体编译中循环索引表达，常用于描述访存模式。

中国知网 https://www.cnki.net

• **Vector Dialect** 作为 Linalg 的主要递降对象，也是重要的用于结构化代码生成的 Dialect，IR(Intermediate Representation)结构设计与 Linalg 类似，不同于传统的 CPU 中的向量指令，Vector Dialect 描述抽象支持多维度的张量类型，需要进一步递降到对应的硬件指令描述。

•**GPU Dialect** 对 GPU 硬件特性进行建模包括线程 id、线程块 id、栅障等，主要用于建模深度学习模式中 GPU 上算子 kernel 的计算和访存模式。

• **Arith** 和 **Math Dialect** 用于表示整数或者浮点数计算。通常作为 Linalg 和 Vector 等 Dialect 的负载，并支持各种抽象层级的类型包括张量、向量、标量。

•**LLVM、SPIRV Dialect** 处于 MLIR 编译系统中的边缘位置，属于 low-level Dialect，用于转换到外部工具链如 LLVM 编译器，进行后续代码生成操作。

• **SCF** 和 **CF Dialect** 处于 low-level Dialect，用于描述模型中的结构化控制流操作，比如常见的循环 for 和条件分支操作 if、while 等，依旧采用计算负载域设计，标明循环和负载的边界，助于简化分析和变换。

理想情况下特定的深度学习编译器只需要组合现有的 Dialect，并加入自己的扩展或者定制，相对于组合来自于各种层次的操作而言，这是更粗粒度的一种构建方式。每个 Dialect 都包含一套协调的操作和类型，这种构建方式更加易于管理，会使得软件栈的结构层次更加清晰。

2.2 模型量化

本节首先对本文的研究对象——深度学习模型量化技术进行简单介绍，接着研究了当前主流的 FP16、INT8 类型量化方案，然后对 FP8 类型展开介绍。

2.2.1 模型量化简介

随着深度学习模型的迅速发展，网络模型变的越来越庞大，对于计算和存储资源有限的设备来说，模型部署变得越来越具有挑战性。模型量化[29]是深度学习模型部署过程中一种通过降低模型数据位宽、减小模型规模以及提高模型推理效率的优化技术。一些深度学习模型对数据的精度要求并不高，因此采用低精度推理可以有效节省内存空间的占用，同时又能保持较高的模型精度。在低精度数据类型中，FP16 类型已经被大多数硬件厂商默认支持，而 BF16 和 INT8 类型的应用也日益广泛。以 INT8 类型为例，其数值范围在区间上均匀分布，由

中国知网 https://www.cnki.net

于值域较小，需要依赖量化校准算法对原始数据进行缩放，以使得量化后数据能在一定程度上近似网络模型中的原始数据分布。然而，当原始数据中出现离群点时，量化效果可能并不理想。英伟达、Graphcore 等厂商提出了符合高斯分布的 FP8 格式[30]，这种格式更符合网络模型中数据的分布特点。由于 FP8 数据格式的多样性以及不同网络中数据值域的差异，FP8 量化效果也有所不同，因此，当前 FP8 量化技术仍处于不断发展阶段。

模型量化分为训练后量化(Post-Training Quantization, PTQ)和量化感知训练(Quantization Aware Training, QAT)两种。QAT 技术通过在模型训练中引入量化过程，能够让模型参数更好地适应量化带来的信息损失，从而提高网络精度。在端侧部署场景中，模型的训练和部署通常分开进行，二者在计算能力和内存需求上存在较大的差异。PTQ 针对已经训练好的模型进行量化，不需要大量的数据参与量化过程，将训练过程与模型量化分离，缺点是量化后模型的精度稍差于 QAT。由于 QAT 更易于部署并被广泛使用，在实际应用中更受欢迎。在模型量化的过程中，需要利用模型中算子的输入输出的类型信息进行量化推理，一些框架如 PyTorch 使用 Python 语言对深度学习模型进行建模，在运行时进行类型推导以获取量化相关信息，这样会失去一些提前优化的机会。为了解决信息获取和优化的问题，可以利用编译器的中间表示来为模型量化提供信息，并进一步分析内存占用情况以进行优化。

近些年，深度学习领域涌现了一批代表性的编译器，如 XLA、TVM 以及 MLIR 等。这些编译器不仅具备深度学习领域专有的优化方法以及中间表示分析技术，还具备传统编译器的优化技术。能够将来自不同框架的深度学习模型在多种后端硬件上快速部署。其中 MLIR 编译器具备良好的可拓展性，能够快速组建一套满足需要的编译流程，以实现定制化的编译优化与代码生成。在 MLIR 编译器将高级 IR 转换为低级 IR 时，可以进行类型转换、代码生成，以生成量化需要的类型信息，为量化技术的实现提供支撑。

随着深度学习模型与人工智能加速器的不断更新迭代，诸如 Google、IBM 等厂商纷纷采用新的数据类型对模型进行加速，并设计出符合身份需求的格式以取得深度学习模型部署性能和精度的平衡。目前，主流的量化类型包括 FP16、BF16、INT8 等。其中 FP16、BF16 浮点格式与 FP32 格式类似，只是在阶码的位数和尾数的位数上有所调整。以 FP32 格式为例，其包含 1 位符号位、8 位阶码位和 23 位尾数位，计算公式如公式 2.1 所示。

中国知网 https://www.cnki.net

$$f = (-1)^s 2^{p-b}\left(1 + \frac{d_1}{2} + \frac{d_2}{2^2} + \cdots + \frac{d_m}{2^m}\right) \tag{2.1}$$

其中 s 为符号位、p 为阶码位、b 为阶码偏移值、d 表示尾数。FP32 数据类型具有较广的表达范围和较高的精度，在深度学习模型的训练和推理中表现出非常好的效果。由于其需要较大的内存空间和计算量，并不适合在边缘设备上进行模型部署。通过模型量化，可以将 FP32 模型以较低位宽的数据格式进行计算和存储，同时能够近似 FP32 模型的计算效果，量化过程如公式 2.2 所示。

$$r = S(q - Z) \tag{2.2}$$

其中 r 表示 FP32 类型的原始数据、q 表示量化后的低位宽数据、z 表示偏移值、S 表示缩放因子。通过选择合适的缩放因子 S 与偏移值 z 可以很好的近似 r 值。

• FP16、BF16 量化

IEEE 浮点运算标准[32]于 2008 年引入了 FP16 存储格式。FP16 包括 1 位符号位、5 位阶码位和 10 位尾数位。相较于 FP32 类型，FP16 在数据精度和表达范围上都有所减少，但在大多数深度学习模型中能够实现精度无损失，并且能够同时减少网络模型内存占用量达 50%。

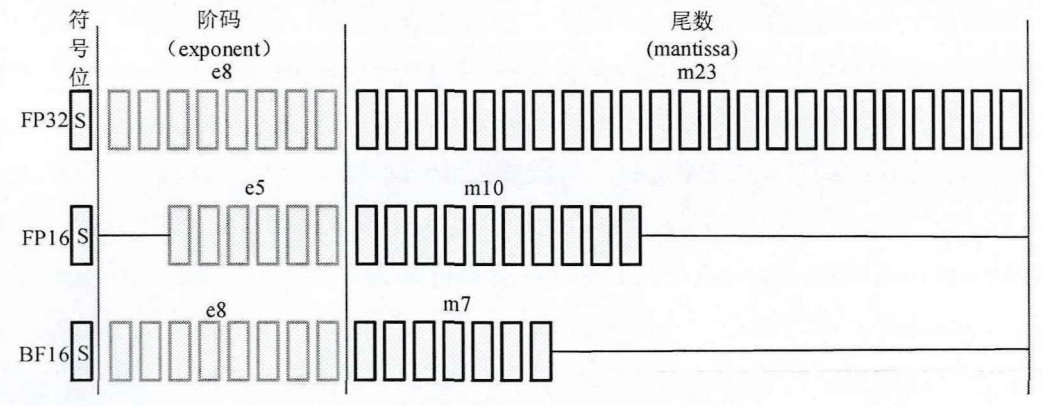


图 2.3 FP32、FP16 与 BF16 格式对比

FP16 数据类型的最大正数为约为 65504。由于一些深度学习模型在训练中对精度要求低但对表达范围要求更高，Google 团队提出了 BF16 浮点格式[33]如图 2.3 所示。BF16 的表示范围与 FP32 类型相同，但尾数只有 7 位，因此在精度上略低于 FP16。根据量化公式 2.2 所示，FP16 与 BF16 量化中的缩放因子 S 通常选取为 1.0，z 选取为 0。

• INT8 量化

中国知网 https://www.cnki.net

INT8 类型量化是 Google 在 TensorFlow Lite 中采用的量化方案。将 FP32 类型数据转换为 INT8 类型，相当于对数据进行 8bit 整数再编码，根据是否使用符号位，可以将 FP32 类型的原始数据映射到-127~127 或 0~255 区间。INT8 量化的关键在于选择适当的缩放因子，即公式 2.2 中的 S，常用相对熵(KL 散度)来衡量 INT8 量化后的数据和 FP32 原始数据分布之间的相似度。INT8 的量化流程主要包括以下两个步骤：

（1）通过 KL 散度算法进行模型校准，计算出对称量化所需的缩放因子 S。

（2）根据量化公式 2.3 进行 INT8 量化推理，使用 8bit 整数计算近似 FP32 浮点数计算，其中 *round* 函数用于对缩放后数据进行取整。

$$q = round\left(\frac{r}{S} + Z\right) \tag{2.3}$$

INT8 量化需要完整的模型校准流程，其值域在区间上均匀分布，对于一些具备较多离群点的模型，量化效果可能不佳。通过选择合适的校准算法可以在大多数模型上获得较好的精度。

2.2.2 FP8E4M3 和 FP8E5M2

FP8 在节省模型的内存占用方面与 INT8 类似，但 FP8 具备小数位，相比 INT8 在数据表示上具有更高的精度。与 FP32 不同的是，FP8 类型并没有 IEEE 规定的阶码和尾数的分配方式，也没有标准的阶码偏移值限制，因此具有较强的灵活性。在 2022 年，英伟达联合 AMD、ARM 等公司发布了 FP8 格式标准，其中包括 FP8E4M3 和 FP9E5M2 两种格式，如下图 2.4 所示。

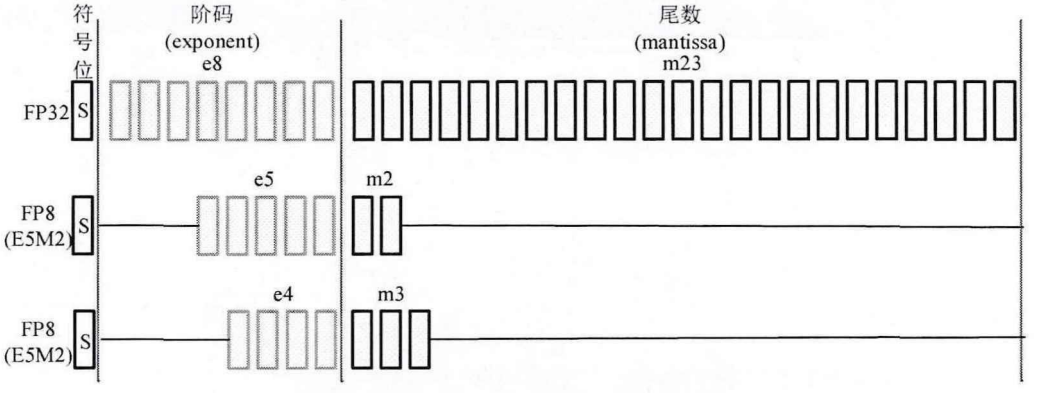


图 2.4 FP32 与 FP8E4M3、E5M2 格式对比

FP32 数据对于零值、NaN、无穷值、非规格化等特殊数值都有着对应的二

中国知网 https://www.cnki.net

进制编码格式，由于 FP8 类型仅使用 8 位比特位与 FP32 的表达能力存在一定的差别。FP8 数据类型的 E4M3 和 E5M2 两种格式，主要区别在于 1 位二进制是分配给阶码还是尾数，这造成了表示范围和精度上的差异。FP8E4M3 具有 4 位指数 3 位小数，二进制编码如表 2.1 所示。FP8E4M3 格式具有 4 位阶码和 3 位尾数，其二进制编码如表 2.1 所示。

表 2.1 FP8E4M3 二进制编码规则

值类别	二进制表示
阶码偏置值	7
无穷值	无
NaN	$S.1111.111_2$
零值	$S.0000.000_2$
规格化最大值	$S.1111.110_2=1.75*2^8=448$
规格化最小值	$S.0001.000_2=2^{-6}$
非规格化最大值	$S.0000.111_2=0.875*2^{-6}$
非规格化最小值	$S.0000.001_2=2^{-9}$

表格 2.1 中展示了 FP8E4M3 编码的详细规则，其中阶码的偏移值固定为 7，能够表示零值和 NaN，但无法表示无穷。由于 FP8E4M3 中的 NaN 使用了两种编码进行表示，存在空闲的二进制编码。

表 2.2 FP8E5M2 二进制编码规则

值类别	二进制表示
阶码偏置值	15
无穷值	S.11111.002
NaN	S.11111.{01,10,11}2
零值	S.00000.002
规格化最大值	S.11110.112=1.75*215=57344
规格化最小值	S.00001.002=2-14
非规格化最大值	S.00000.112=0.75*2-14
非规格化最小值	S.00000.012=2-16
非规格化最小值	S.00000.012=2-16

为了充分利用 FP8E4M3 数制的所有编码，规定当阶码位全为 1，尾数不全

中国知网 https://www.cnki.net

为 0 时表示正常值，从而为 FP8E4M3 拓展出了额外的七个数值(256，288，320，352，384，416，448)，因此 FP8E4M3 数制的表示范围为[-448，448]。此外，FP8E4M3 还支持非规格化数值表示，当指定隐藏的整数位为 0，阶码值固定为-6，能够表示最小绝对值 2^-9。

表格 2.2 中展示了 FP8E5M2 数制的详细编码规则，FP8E5M2 与 IEEE754 规定的浮点标准更加接近。其中指明阶码的偏移值为 15，能够表示零值、无穷和 NaN 数据。与此同时，FP8E5M2 也具备非规格化数值表示能力，当指定隐藏整数位为 0、阶码值固定为-14，其能够表示最小绝对值为 2^-16。

2.3 算子融合优化与硬件架构

2.3.1 算子融合优化技术

算子融合是深度学习编译器中推理加速的关键技术之一。在算子融合技术研究的早期阶段，硬件厂商针对自家硬件架构特征来实现高性能算子库，为了提升算子库的性能，充分的利用硬件，开发人员手动将多个算子合并为一个更大的算子，手动算子融合根据具体的硬件特征和算法来设计融合策略，以最大化提升程序性能。MLIR 编译器同时具备算子融合和融合后算子生成的能力，算子融合是在计算图的基础上，对计算图中的算子进行标记，不同的组表示不同的算子，在算子生成阶段会以组为单独进行完整的算子生成，在算子生成的过程中对组内的算子进行分块、内联、重排、循环交换和融合等操作以达到新算子性能的提升。通过算子融合可以充分利用寄存器和缓存，降低计算的中间结果在多级存储体现中的重复搬运，在垂直算子融合中，前一次的计算结果可以暂存在寄存器和缓存中，下一次计算可以直接从寄存器或缓存中读取，减少了内存读写的 IO 次数，同时在 GPU 中进行算子融合，可以减少核函数的调用开销。图 2.5 中展示了从主机端调用核函数的耗时情况，A 表示主机调用核函数，B 点表示核函数启动，C 点表示核函数启动控制返回主机线程，D 点表示核函数执行完成，E 点表示主机同步核函数。在 GPU 平台算子融合可以减少 B 到 C 之间的耗时，并可以通过循环融合等技术缩短 C 到 D 点之间所消耗的时间。

不同于垂直算子融合，水平融合主要用于融合那些在模型中处于水平位置关系的算子，这些算子往往计算量小或访存密集，将这些算子融合可以减少 B 到 C 的内核启动开销，同时可以通过优化融合后的内核如提升 GPU 并行度，增大计

中国知网 https://www.cnki.net

算负载，提升整体性能。

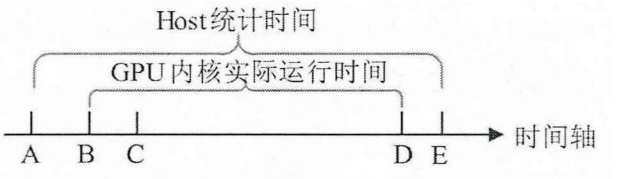


图 2.5 主机端调用核函数运行过程

2.3.2 CPU+GPU 异构平台

在 CPU 和 GPU 构建的异构平台中，CPU 负责执行程序中的数据流和控制流，同时能够控制和调度 GPU 设备，当需要时启动 GPU 设备将部分程序和数据传输到 GPU 端进行执行。GPU 具备自身的硬件架构，包括计算单元，存储单元，内存控制单元和任务调度单元，可以看作一个独立的系统，在异构环境中，GPU 通过总线和 CPU 处理器进行数据传输，以提高程序的执行效率，获得更好的加速效果。GPU 整体硬件架如图 2.6 所示，主要由下面几个部分组成：

（1）计算单元阵列，如图 2.6 中 SM(Stream Multiprocessor)以及 SM 中的 CUDA(Compute Unified Device Architecture) Core 等；

（2）多级缓存系统如 SM 间共享的 L2 缓存；

（3）全局内存、共享内存、寄存器等；

（4）CPU 与 GPU 间的数据通路、GPU 间互联接口 NVLink；

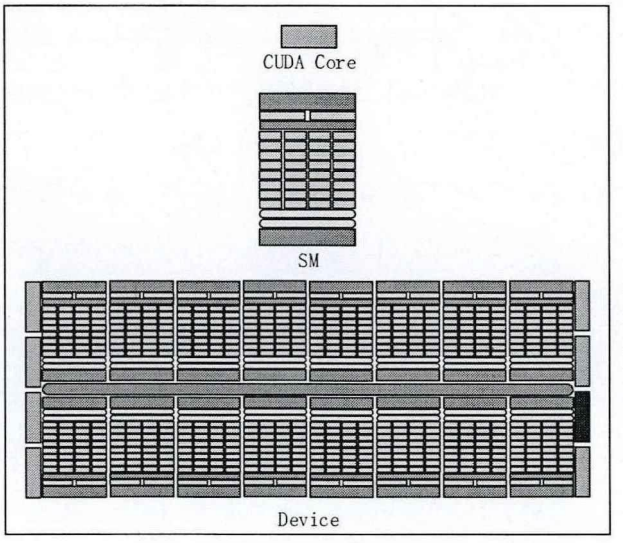


图 2.6 GPU 整体硬件架构

中国知网 https://www.cnki.net

GPU 硬件主要由流多处理器单元 SM 组成，是 GPU 中核心的计算与存储资源，每个 SM 中包含大量的 CUDA 核心用于启动线程参与计算，这些 CUDA 核心基于 SIMT(Single Instruction Multiple Thread)进行计算，每个 CUDA 核心具有一定独立性具备自身的寄存器、计算单元和数据缓存等硬件。在计算时，每个 CUDA 核心在相同的指令和不同的数据下并行执行各自的任务。GPU 的吞吐量非常高，一块 GPU 卡可以拥有几千个流处理单元，如果任务有足够的线程，GPU 可以通过核心间并行计算快速完成它。GPU 资源通过 CUDA 编程模型进行调度，CUDA 编程模型主要由 Grid、Block、thread 等组成，如图 2.7 展示了 CUDA 编程模型的资源组织方式。

CPU 通过启动 GPU 内核，将程序分派到 GPU 上进行执行，内核程序需要被进一步划分到更细粒度的单位进行并行执行，GPU 使用 CUDA 编程模型对硬件资源进行调度。

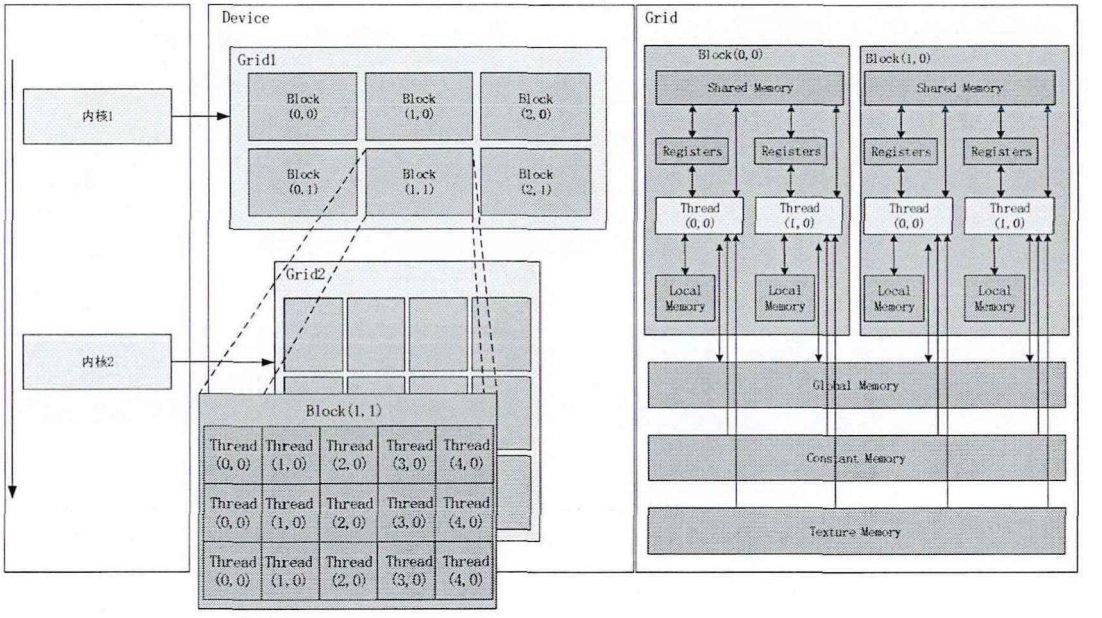


图 2.7 CUDA 编程模型

CUDA[34]是英伟达 GPU 软件开发套件，主要用于在深度学习模型训练和推理中对模型中的算子或者函数进行加速。CUDA 提供了 grid、block、warp、thread 等抽象，其中使用 thread 线程表示 SM 中的每个 CUDA 核心；warp 指线程束，一般包含 32 个线程，是 GPU 的最小调度执行单位；block 表示线程块，是对线程的组织形式；grid 表示线程网格，是更高层级的抽象表示对线程块 block 的组

中国知网 https://www.cnki.net

织形式。一般的，GPU 内核在启动前需要指定线程网格数、网格内线程块数、线程块内线程数以指定 GPU 启动硬件资源参与并行计算。

GPU 拥有独立于 CPU 内存的存储体系，用于存储线程在计算时所需要的数据，包括全局内存、纹理内存、共享内存、本地内存、寄存器等存储单元。一般的，CPU 在启动 GPU 内核前，需要将内核需要的数据搬运到 GPU 的全局内存，全局内存就是常说的显存，是最常使用的内存，拥有 GPU 上最大的存储空间，但延迟较高，所有线程都可以访问。对于一些空间局部性比较好的程序，GPU 提供了可编程的共享内存，通过将全局内存中的数据加载到共享内存中，线程可以更快的传输并处理数据。相对全局内存，共享内存的空间相对较小，延迟更低，带宽也更高，但共享内存仅限线程块内的线程共享，不同线程块间互相独立。寄存器是 GPU 是速度最快的内存空间，通常用于保存被频繁使用的私有变量，属于线程私有，在 GPU 中每个线程具备的寄存器数量是有限的，当内核使用了过多的寄存器，数据就会从寄存器溢出到本地内存，本地内存属于线程私有，但速度与全局内存一致，因此寄存器溢出会损失程序性能。

CPU 与 GPU 不同的架构设计特点表明它们适合处理不同类型的程序，在异构编程环境中 CPU 主要用于处理复杂数据流和程序控制流，对 GPU 进行调度以提升程序整体性能。GPU 善于多核并发，将所有线程都被分配到真实的计算核心进行处理。一个 GPU 的计算核心可以拥有数百到数千个计算单元，这种硬件特点使得 GPU 非常适合进行计算密集型任务，同时 GPU 具备多级内存系统，需要对计算任务进行合理划分，选择合适的 block、thread 数以使得程序获取更好的性能。

2.4 本章总结

本章节首先从 MLIR 编译器的设计以及其内置 Dialect 介绍了本文的研究平台，接着对 MLIR 编译器开展的深度学习模型量化技术以及 FP8 数据类型进行阐述。最后介绍了深度学习编译器中的算子融合技术以及模型部署的异构硬件平台，为后续基于 MLIR 编译器开展 FP8 量化和算子融合优化奠定了基础。

中国知网 https://www.cnki.net

3 基于 MLIR 的 FP8 量化策略支持

当前 MLIR 编译器量化系统仅支持 FP16、INT8 类型，为了使得量化系统更加完备，同时评估模型在新的 FP8 数据类型的量化效果，为用户在模型压缩部署时根据需要提供最佳选择，本章在 MLIR 编译器下对 FP8 数据类型量化进行支持。本章首先介绍了 FP8 量化策略的研究动机，接着从值转换规则构建、量化类型与属性构建、缩放因子选取三个方面描述了 FP8 量化模拟策略方法的实现过程，最后通过实验验证了该量化方案的部署难度与模型精度。

3.1 研究动机

FP8 数据类型是由英伟达、Arm 等企业于 2022 年提出的新数据格式，属于浮点数据类型，当前 LLVM 编译基础设施已经在 APFloat 模块加入对 FP8E4M3 和 FP8E5M2 类型的支持，为 FP8 类型加入 MLIR 编译器中提供了基础。FP8 数据类型在空间占用上与 INT8 类型相同只有 8bit，但 FP8 数据类型的格式相对灵活，可以在精度和表达范围间调整。既往的研究如 Sun 等人于 2019 年提出 HBF8 指数偏移值设置为 11，偏移值过大带来较小的表达范围，对于一些动态范围比较大的网络，精度下降明显。英伟达联合 ARM、AMD 等发表 FP8 格式标准论文中缺乏部分深度学习模型的测试同时未能对量化缩放因子进行探究，因此为了更为全面的评估 FP8 数据类型在深度学习模型推理中的量化效果，同时因市面缺少支持 FP8 的硬件，考虑对深度学习模型进行 FP8 量化模拟研究。

选择在 MLIR 编译系统中对该量化类型进行支持，一方面是由于该系统具备对 FP16 和 INT8 的量化能力，具备研究对比意义，同时缺乏对 FP8 数据类型的量化支持。另一方面，该系统使用了 MLIR 编译器内置的量化模块，具备一定的可拓展性，因此考虑在该平台开展研究，通过增加 FP8 类型量化支持可以为模型压缩部署提供更多的选择空间。

3.2 实现方法

本节首先研究了 FP8 两种数制对应的数据类型与 FP32 数据类型间数值的转换算法，根据 MLIR 内置量化模块进行拓展以在 IR 层面和运行时支持 FP8 数据

中国知网 https://www.cnki.net

类型的变换和推理。接着深入编译流程在关键算子比如卷积的 IR 表达中添加缩放因子属性，帮助在运行时进行量化模拟推理，最后通过对应模型的评估算法对量化后模型的 IR 中间表示进行准确性评估。

3.2.1 值转换规则构建

在神经网络的推理过程中，FP32 类型的数据可能会产生零值、无穷值、NaN 和非规格化数值，这些特殊数值都存在着对应的二进制格式。FP8 数据类型使用 2022 年英伟达联合 AMD 等厂商发布的 E4M3 和 E5M2 两种格式进行编码，它们区别在于指数和小数位的分配，FP8E4M3 格式具有 4 位指数和 3 位小数，具体二进制编码见表 2.1 所示。

表格 2.1 中展示了 FP8E4M3 编码表的规则细节，其中阶码偏移值为 7，可表示零值和 NaN，但不能表示无穷值。FP8E4M3 的表达范围为[-448, 448]。同时 FP8E4M3 支持非规格化数值表达。 表格 2.2 中详细展示了 FP8E5M2 的编码规则。相比 FP8E4M3，FP8E5M2 与 IEEE754 浮点标准更加接近。其中阶码的偏移值为 15，使得它能够有效表示零值、无穷和 NaN 数据。同时，FP8E5M2 可以表示非规格化数值。对比 FP8E4M3 的编码规则，FP8E5M2 拥有更广的表达范围和更强的特殊值表示能力。然而，由于 FP8E5M2 仅有 2 位小数位，导致其数值分布更为离散，降低了表达精度。为了将 FP32 中数值表示转换为 FP8E4M3 和 E5M2 两种格式的编码表示，需要对 FP32 数据进行分类处理，主要划分为以下三类:

（1）FP32 数据值处于 FP8 表示范围之中。

（2）FP32 数据值小于 FP8 表示最小绝对值。

（3）FP32 数据值超出 FP8 表示的最大绝对值。

为了提高数值转换效率，减少不必要的值域判断，根据 FP32 数据格式中的指数位（即阶码）进行分类，而不是根据数据范围进行区分。IEEE FP32 数据类型具有 8 位阶码，表达范围为 0 到 255，阶码的偏移为 127，实际的阶码范围为 -127~128。以 FP32 数值转换为 FP8E4M3 数值为例，E4M3 数据具有 4 位阶码，表达范围为 0~15，阶码的偏移为 7，实际的阶码范围为-6~7。因此，二者的转换可以分为以下四类：

（1）FP32 数据的真实阶码处于 FP8E4M3 真实阶码表示区间。

（2）FP32 数据的真实阶码小于 FP8E4M3 真实阶码的最小值。

中国知网 https://www.cnki.net

（3）FP32 数据的真实阶码大于 FP8E4M3 真实阶码的最大值。

（4）FP32 数据中的特殊值如无穷、零值和 NaN 转换到 FP8E4M3 表示。

对于 FP32 数据中真实阶码小于 FP8E4M3 最小阶码的情况要进一步分类。如图 3.1 所示，FP8E4M3 中的非规格化数据可以用于近似表达 FP32 数据中较小阶码的规格化数值，而对于小于 FP8E4M3 非规格化数据的 FP32 数据，默认使用 FP8E4M3 中的 0 表示。

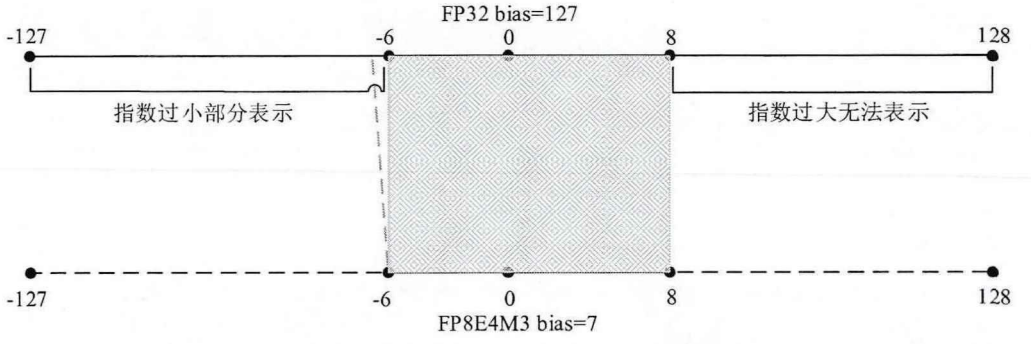


图 3.1 FP32 到 FP8 的映射过程

在将 FP32 数据转换到 FP8 时，由于 FP8 的阶码和尾数范围更小，因此必然会导致数据损失。然而，在将 FP8 数据转回 FP32 时，所有 FP8 中的特殊值都可以被 FP32 准确表示。对于 FP8 数据中的规格化浮点值，在转换回 FP32 时，只需调整指数之间的差值即可完成转换。以 FP8E4M3 转回 FP32 为例，公式 3.1 展示了二者间的阶码计算方式，而对于 FP8E5M2 类型，只需将公式 3.1 中的偏移值 7 更改为 15 即可实现转换。

$$E_{FP32} = E_{FP8} - 7 + 127 \tag{3.1}$$

对于 FP8 表示的非规格化数据，FP32 同样可以完全表示，但要注意 FP32 使用规格化数值表示 FP8 非规格化数值，具体的转换过程如算法 3.1 所示。对于 FP8 表示的非规格化数据，同样可以被 FP32 完整表示，但需要注意，FP32 在表示 FP8 的非规格化数据时采用规格化的数据格式，具体的转换过程如算法 3.1 所描述。

在算法 3.1 中，减去偏置常量 bias 的原因是 FP8E4M3 表示的是非规格化数值，其尾数之前整数位默认为 0。当将其转换回 FP32 时，FP32 中规格化数据的尾数之前整数位默认为 1，这导致转换后的 FP32 数值比原始的 FP8 数值更大。

中国知网 https://www.cnki.net

因此，为了将其恢复为 FP8 格式表示的原始值，需要减去一个偏置值。

算法 3.1： FP8E4M3 数值转换回 FP32

Input: 所有的 FP8E4M3 格式数据 FP8Data

Output: 对应的 FP32 格式数据 FP32Data

```
1 for all FP8Data do
2    w = FP8Data << 24;
3    sign = w & 0x80000000;
4    exponent = w + w;
5    /*将FP8Data规格化值转换为FP32规格化值*/
6    normal_v = exponent>>5 + 0x3C000000;
7    mask = 141 << 23;
8    bias = 0x1.0p+14f;
9    /*将FP8Data非规格化值转换为FP32规格化值*/
10    cutoff = 0x10000000;
11    if exponent < cutoff then
12       FP32Data = sign | denormal_v;
13    else
14       FP32Data = sign | normal_v;
15    end if
16    return FP32Data;
```

3.2.2 量化类型与属性构建

为了在 MLIR 的中间表示文件中表示具体的量化信息，需要借助 MLIR 编译器内置的 Quant Dialect 模块。该模块专注于量化实现，提供了支持量化所需的数据类型、数据的类型转换以及类型校验系统。例如，CalibratedQuantizedType 用于表示校准类型，用于在中间表示转换过程中携带当前网络层激活值的 threshold 阈值或最值数据，如图 3.2 所示。在此过程中，FP32 表示量化校准所使用的数据类型是 FP32，括号内的数值表示当前层输入或输出值的最值。如果这两个值相等，则用于计算对称量化缩放因子 S 中的 threshold。

此外，还有用于表示均匀量化类型的 uniformQuantizeType。该类型由原始数据的表达类型和数据量化后的存储类型两个部分组成。其中原始数据的表达类型支持 FP32、FP16、BF16 等，而存储类型支持有符号或无符号的 INT8 类型，

中国知网 https://www.cnki.net

以及其他宽度的整型。在均匀量化类型类型中，Scale 是一个双精度值，而 Zeropoint 则为一个整数值，用于表示量化偏移。

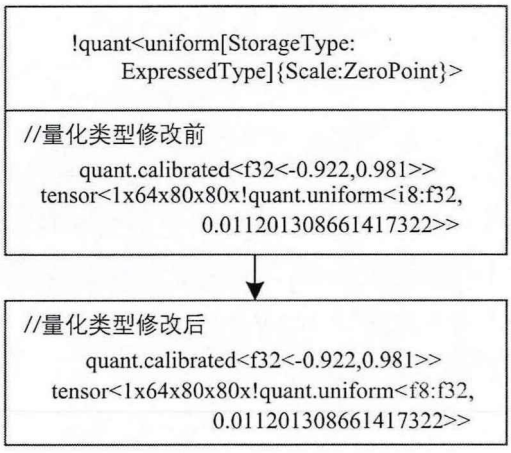


图 3.2 FP8 量化类型构建

由于 FP8 类型作为一种新兴类型，出现时间较晚，MLIR 中量化模块开发时未能考虑到新类型的出现。因此 MLIR 量化模块中并不支持将 FP32 表达类型转换为 FP8 存储类型。本文为了实现对 FP8 类型的量化支持，深入修改了 Quant Dialect 中的量化类型系统以及校验系统。为了保持最小侵入式改动，复用并拓展了均匀量化类型的存储类型支持，修改了类型解析器系统以满足 FP8 类型的解析，修改了 IR 打印器系统中对量化类型 FP8 的处理，在 IR 层面支持 FP32 到 FP8 的量化类型表达，如图 3.2 所示。

```
%5 = "tpu.Conv2D"(%2, %3, %4) {dilations = [1, 1], float_scale =
[0.05740785226225853, 0.0045583001337945461…], group = 1 :
i64, kernel_shape = [6, 6], kernel_zp = 0 : i64, pads = [2, 2, 2, 2],
strides = [2, 2], with_bias = true} :
(tensor<1x3x640x640x!quant.uniform<f8:f32,0.0022321607142857
>>, tensor<32x3x6x6xf32>, tensor<1x32x1x1xf32>) ->
tensor<1x32x320x320x!quant.uniform<f8:f32,0.16413830758928>>
```

图 3.3 量化属性构建

为了评估更细粒度的量化效果，在 MLIR 中实现对 FP8 类型的逐通道量化技术。当前 MLIR 将权重数据文件对象和深度学习模型建模在同一层 IR 中。这样有利于在模型推理之前计算出卷积核每个通道的缩放因子，从而减少推理时的

中国知网 https://www.cnki.net

延迟。为了将计算出的缩放因子提前存储在中间表示中，对卷积算子的 IR 属性进行了扩展，添加了一个名为 float_scale 的浮点格式属性。修改后的卷积算子 IR 如图 3.3 所示，考虑到权重通道较多，此处仅展示部分通道的缩放因子值。

3.2.3 缩放因子选取

根据公式 2.3 可得知，缩放因子和零点值的选择直接影响了量化后模型的精度。其中，零点值是根据低位宽数据类型的分布区间来确定的。由于 FP8 类型的数据的分布特征与 FP32 类型类似，并不需要对数据进行偏移，因此将零点值设定为 0。为了全面评估缩放因子对 FP8 类型模型量化的精度影响，本文将 FP8 缩放因子的选择过程分为三个步骤：

首先，将缩放因子的默认值设定为 1.0，即不对模型中的 FP32 数据进行缩放，这样能够直接观察由类型转换误差对模型精度产生的影响。接着以 1.0 为基准，选择 0.1、0.2、0.25 等值进行测试。最后，为了充分考虑深度学习模型中各个层的激活值差异，采用 Percentile99.99%校准策略计算所需的 threshold 值，如公式 3.2 所示。以 FP8E4M3 为例，其中 Qmax 和 Qmin 表示量化后数据的最大最小值±448。

$$S = \frac{threshold}{Qmax - Qmin} \tag{3.2}$$

相比针对激活值的逐层量化方案，面对权重数据的逐通道量化粒度更细，可以更好地适应不同通道之间的数据分布差异，提供更准确的量化结果。本文使用最大值策略对当前通道内数据的最大值进行统计，将统计值作为当前通道的 threshold，通过与 FP8E4M3 类型的最大值 448 相除得到最终的缩放因子值。此外，选择数值计算中常使用的 ROUNDING_HALF_TO_EVEN 方法作为取余策略。

为了在 MLIR 中间表示层面生成新的 FP8 量化数据类型，借助 MLIR 系统中的 Dialect Conversion 过程来完成这一步骤。通过对中间表示的递降，生成带有从 F32 到 FP8 量化类型算子的中间表示。图 3.2 展示了从 FP32 类型转换为量化校准类型，然后进一步转换为 FP8 量化类型的过程，整体的 FP8 量化模拟策略如公式 3.3 所示。

$$x_i^{(q)} = ToFP32\left(S * ToFP8\left(x_i^{(S)}\right)\right) \tag{3.3}$$

其中

中国知网 https://www.cnki.net

$$x_i^{(S)} = clip\left(round\left(\frac{x_i}{S} + z\right), -448, 448\right) \tag{3.4}$$

x_i表示 FP32 类型的原始数据值，S表示缩放因子，z为零点值，这里采用对称量化，因此z取 0，clip 为裁剪函数，裁剪范围为以 FP8E4M3 为例为-448 到 448，最后 round 表示取余函数。ToFP8 函数为 3.2.1 章节中构建的从 FP32 类型到 FP8 类型的转换规则。ToFP32 函数表示从 FP8 类型转换回 FP32 类型的算法 3.1 实现。使用缩放因子能够在一定程度上提升模型的准确度，这是因为一些深度学习模型中的激活值或权重数值较小，过小的数值在转换为 FP8 类型时因无法被表示转换为 0 值。通过缩放因子对 FP32 类型的原始数据进行一定程度的放大，有助于这些数据在量化后保持对模型推理的影响，从而改善精度。FP32 到 FP8 的量化模拟策略如下：

（1）通过小批量模型输入数据并基于 Percentile99.99%策略对模型进行量化校准，计算模型中激活值的 threshold，该步骤将模型的参数类型从 FP32 类型转换为校准类型。

（2）通过 MLIR 的 Dialect Conversion 模块对中间表示进行变换，生成新的具有 FP8 量化类型的计算图，并通过公式 3.2 计算缩放因子S添加到卷积等算子的 float_scale 属性中。

（3）遍历模型的 MLIR 中间表示文件，根据模型中算子间的依赖关系进行逐个算子推理。推理前需要根据缩放因子S，对算子的输入的 FP32 类型数据和权重进行缩放、取余和裁剪，转换到 FP8 格式后转换回 FP32 格式参与推理。

（4）将量化后模型中的激活值与量化前 FP32 类型网络中的值进行逐层比对。

3.3 实验结果与分析

3.3.1 实验环境

本文使用 Ubuntu 系统为测试平台，CPU 使用 AMD R7 5800 型号，最高频率为 2.8GHz，内存为双通道 16GB，软件采用 TPU-MLIR 编译框架的 v1.1.beta.1 版本，其依赖的 MLIR 编译器版本为 6e19eea02bbe7747cfca1f2a13287b9987ab959a 分支，Ubuntu 系统使用 20.04 版本。

图 3.3 展示了基于 MLIR 编译基础设施构建的 TPU-MLIR 编译框架结构，主要复用了 MLIR 编译器以及内置的 Func Dialect 和 Quant Dialect，同时搭建了自

中国知网 https://www.cnki.net

定义的 TOP Dialect 和 TPU Dialect。编译框架前端接收训练后的神经网络模型如 PyTorch、TensorFlow 等框架保存的模型或开源项目 ONNX(open neural network exchange)[35]中已经训练好的模型，通过编译系统对模型进行模型校准、数据切片、流水并行等优化，将模型高效部署到端侧硬件 TPU 上。该编译流程与常见的代码生成编译器略有不同，TPU-MLIR 更专注编译过程中的前端和中端转换，最终模型的推理依赖于 CPU 或 TPU 硬件端的算子库。同时为了便于实现多种优化，TPU-MLIR 在中间表示层次提供了基于 Intel OneDNN(oneAPI Deep Neural Network Library)库[36]的 CPU 推理引擎，方便程序验证。TPU-MLIR 整体工作流程如图 3.4 所示，主要分为 4 步。

（1）选择 TensorFlow Lite、Caffe2 或 ONNX 等框架已经训练好的模型，使用前端转换模块将模型转换为到 TPU-MLIR 内部表示。

（2）模型被转换为 TPU-MLIR 中的高层级中间表示 TOP IR，将深度学习模型架构和权重数据信息建模在同一层，并应用图级别的优化遍对模型进行初步优化。

（3）TOP IR 被进一步转换为 TPU IR 表示，该步骤通过 MLIR 的 Dialet 递降实现，进行 TPU 硬件相关的优化如数据重排，流水并行等。

（4）模型被转换为序列化的 bmodel 文件使用 TPU 推理引擎执行。

TPU-MLIR 编译框架目前支持 FP16、BF16、INT8 类型的深度学习模型量化，基于 TOP Dialect 层进行深度学习模型计算图建模，开展量化校准优化。本文测试的主要深度学习模型为 Yolo 目标识别模型和 Resnet 分类模型，对于 Yolo 系统中适合端侧使用的 Yolov5s[37]进行测试，使用公开的 COCO2014 以及 COCO2017 数据集[38]，因系统性能原因，选择对数据集中的前 5000 张照片进行识别以进行精度验证。同样的，对于分类的 Resnet50 模型[39]选择使用 ImageNet 经典数据集[40]，验证前 5000 张照片的识别精度。

对于深度学习网络模型，采用开源项目 ONNX 模型库中预训练的 Yolov5s 和 Resnet50-v1-7 模型，并进行 PTQ 量化，可避免额外的模型训练工作。在模型推理中，使用 FP32、INT8、FPE4M3 和 FP8E5M2 四种数据类型进行精度验证对比。为了评估 Yolov5s 模型的预测精度，使用 COCO 官方数据集提供的 Python 包 pycocotools 工具进行测试，并计算交并集 IOU 为 0.5 时的精度数据。

中国知网 https://www.cnki.net

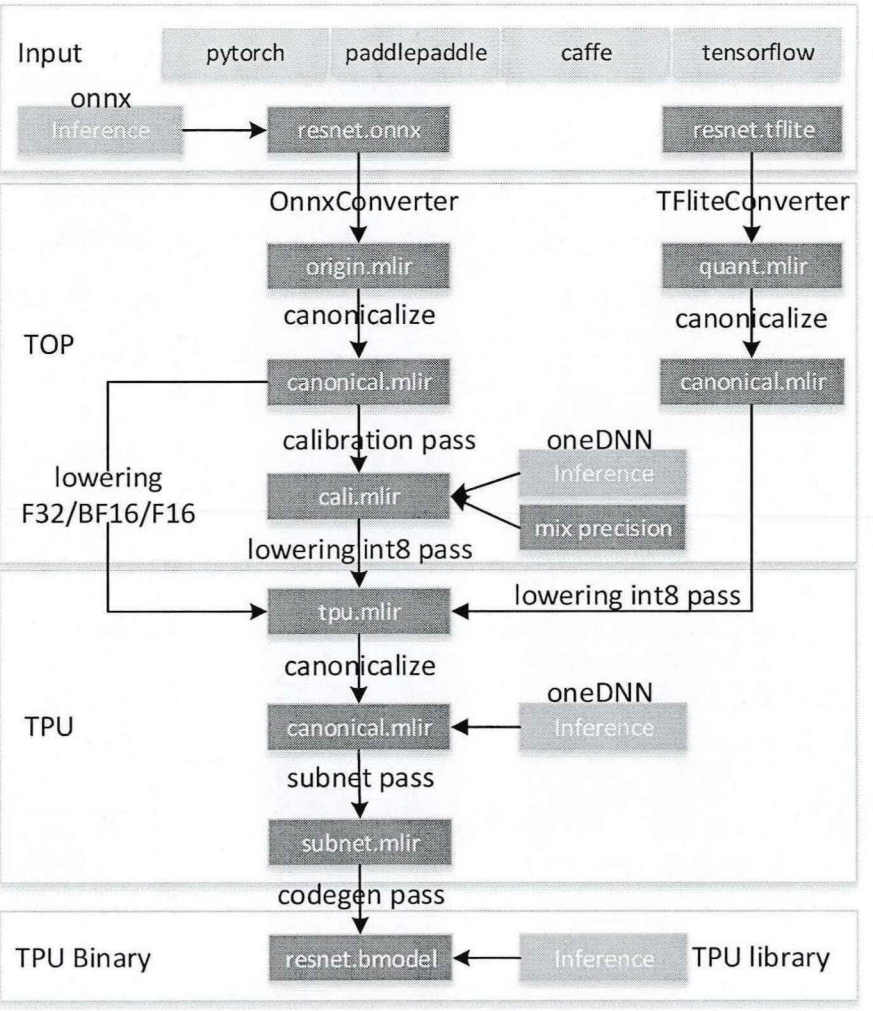


图 3.4 基于 MLIR 的 TPU-MLIR 编译框架

对于 Resnet50-v1-7 模型，采用 ImageNet2012 数据集进行测试，分别统计推理结果中概率最大的标签号（TOP1 准确度）和推理结果中前五个概率最大的标签号(TOP5 准确度),主要对模型中计算密集型算子卷积进行 FP8 量化模拟。为保证实验结果的准确性和精确度，每次测试需要在相同规模的测试集上进行至少 10 次重复实验。

表 3.1 测试集

测试集	输入图片规模	标注文件
COCO2017val	5K	Annotation2017val
COCO2014val	41K	Annotation2014val
ImageNet	50K	分组代替标注

中国知网 https://www.cnki.net

3.3.2 测试结果与分析

实验首先使用了 COCO2017 数据集进行测试，结果如图 3.5 所示。当 FP8E4M3 类型在缩放因子设置为 1.0 时，Yolov5s 的模型精度优于当前系统中的 INT8 量化精度，与 FP32 数据类型相比，模型精度仅下降 0.7%。若将缩放因子调整为 0.2，可进一步提升模型精度表现。FP8E5M2 类型相比 F8E4M3 类型具有更大的表示范围，但尾数仅有两位，精度较低，因此在 Yolov5s 模型量化中精度损失比 F8E4M3 更加明显。当缩放因子默认为 1.0 时，FP8E5M2 量化后的模型精度仅为 52%，较 FP32 类型下降了 4.1%，若将缩放因子设置为 0.1，精度损失可降至 2.3%。

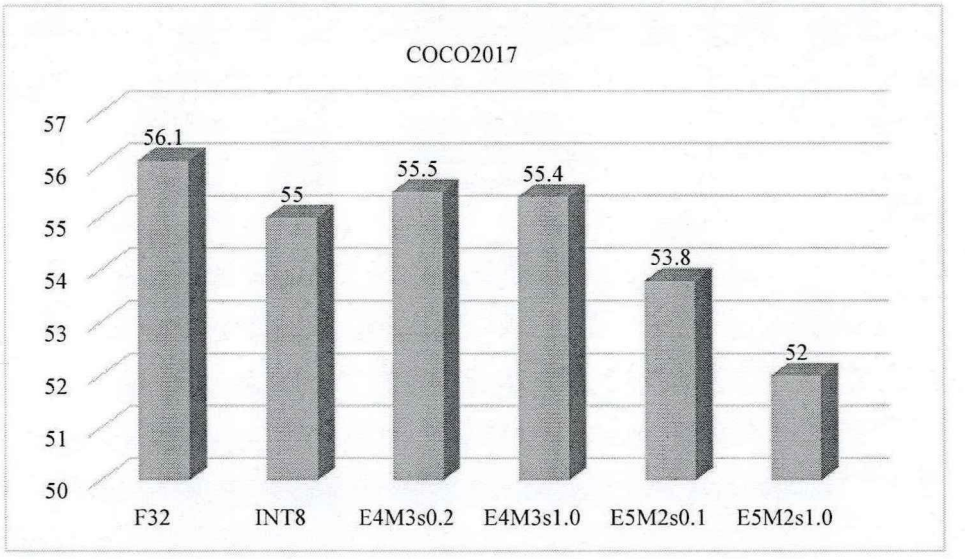


图 3.5 COCO2017 下 Yolov5s mAP 测试 IOU=0.5

在采用 Percentile 99.99%校准策略的情况下，两种 FP8 类型在 Yolov5s 模型中的平均精度分别为 31.3%和 1.2%，远远低于缩放因子为 1.0 的表现。这是由于 Yolov5s 网络中的激活值与权重数据动态范围较小。相对于 INT8 类型，FP8 数据类型在数据较小时数据密集度更高，能很好的模拟原始数据的分布，因此精度也更高。然而，由于 FP8 小数位较少，在较大数值表达上更加离散低，对应的精度也随之降低。根据公式 3.2 的推导可知，缩放因子的选择与 FP8 的表达范围息息相关，较大的表达范围会对应较大的缩放因子。若选择过小的缩放因子会将网络中的数据映射到 FP8 中较为离散的数值，从而带来更大的误差。因此，在该策略下，FP8E5M2 类型与 FP8E4M3 类型的精度表现相对较低。

中国知网 https://www.cnki.net

实验进一步在 COCO2014 数据集上得到的结论也印证了这一趋势，具体表现如图 3.6 所示。在 Percentile 99.99%校准策略下，两种 FP8 格式的模型精度分别为 26.6%和 2.6%。

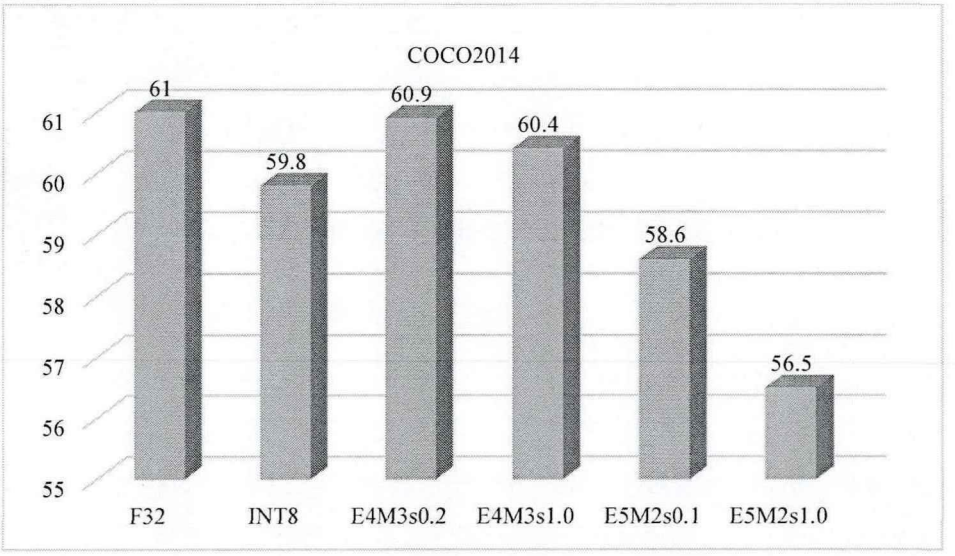


图 3.6 COCO2014 下 Yolov5s mAP 测试 IOU=0.5

对于 ResNet50-v1-7 模型，在经过 ImageNet2012 测试后发现，FP8 两种类型的精度与 INT8 类型非常接近，具体结果如图 3.7 所示。

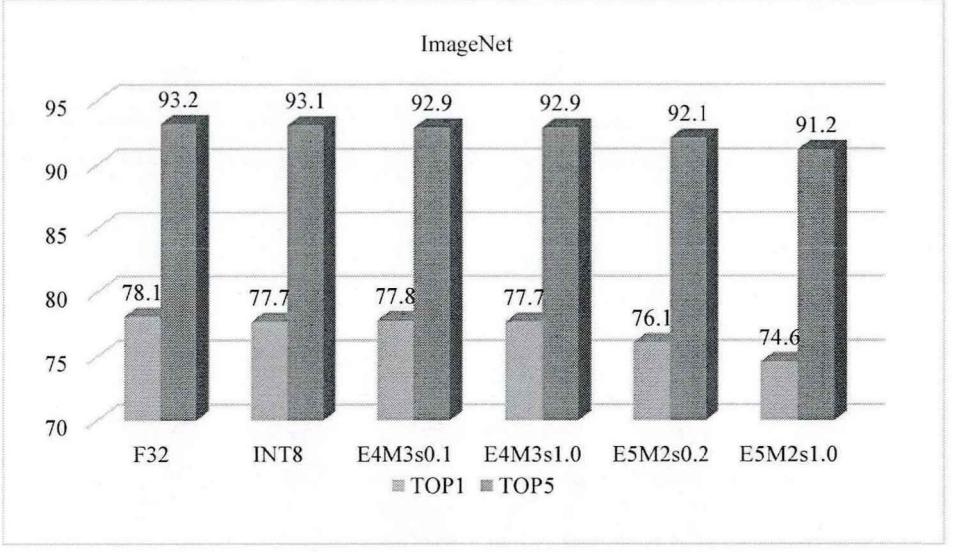


图 3.7 Resnet50 模型精度对比

当 FP8E4M3 类型的缩放因子设定为 0.2 时，模型的 TOP1 准确率略高于 INT8

中国知网 https://www.cnki.net

量化对应的结果。与 Yolov5s 模型的测试结果相似，F8E4M3 类型的模型精度全面优于 F8E5M2 类型。在采用 Percentile 99.99%校准策略后，两种 FP8 格式的模型 TOP1 精度仅为 9.8%和 1.6%，TOP5 精度仅别为 18.1%和 3.8%，远远低于缩放因子为 1.0 时的表现。这主要是因为 ResNet50-v1-7 模型中的数据动态范围较小，将数据扩大到 FP8 类型中较大的数值时产生了较大的误差。

经过对上述实验数据的分析可知，使用 FP8E4M3 类型进行深度学习模型量化的效果略优于 INT8 类型，精度损失在可接受的范围内；而使用 FP8E5M2 类型进行量化时，精度损失整体大于 FP8E4M3 和 INT8 类型。因此面向端侧硬件部署值域较小的模型时，宜选择 FP8E4M3 类型进行量化以获取最佳效果。同时，FP8 类型量化对缩放因子的选取并不敏感，在缩放因子为 1.0 时就能获得较好的模型预测精度。为了达到最佳的模型精度，应以 1.0 为基准进行测试，并可以选择 0.1、0.2、0.25、0.5 等常规值进行测试，无需依赖特定的校准算法。相较于 INT8 量化，FP8 类型省去了模型校准的步骤，使得部署实施更为便捷。该策略在 CPU 端进行推理验证，是一种软件层面的模型量化策略，同样可在支持 FP8 类型的端侧硬件上实现 FP8 类型的模型量化和数制转换工作，因此具备一定的拓展性。

3.4 本章总结

本章基于 TPU-MLIR 编译框架对目标识别网络和分类网络模型网络进行了 FP8E4M3 和 FP8E5M2 量化。研究表明，FP8E4M3 类型的预测精度略优于 INT8 量化精度，相对于 FP32 类型，仅有 0.6%的精度损失。同时发现 FP8 格式对于缩放因子的选取并不敏感，不需要依赖特定的校准算法，可以通过设定如 0.1、0.2 等值进行快速筛选，以获得最佳精度。相较于 INT8 量化方法，FP8 极大地简化了模型部署过程中量化的实施。然而，由于目前支持 FP8 类型的硬件较为有限，例如英伟达 Hopper 架构的芯片难以获得，因此无法在实际硬件上测试性能损失和功耗情况，这也成为未来研究的一个重要方向。总的来说，FP8 的引入为推动模型量化研究提供了新思路，值得进一步研究。

中国知网 https://www.cnki.net

4 基于 MLIR 的推理引擎内存优化

深度学习模型的部署往往需要借助特定的硬件设备来达到高速推理，如 CPU、GPU、NPU 等，这个过程就涉及到存储设备的申请、释放以及内存对齐等操作，这在推理过程中需要一定的时间开销。推理引擎的内存优化，通常是减少频繁的设备内存空间的申请以尽量做到内存的复用。

一般的，可以根据张量生命周期来申请空间：

- 静态内存分配：如一些固定的算子在整个计算图中都会使用，此时需要再模型初始化时一次性申请完内存空间，在实际推理时不需要频繁申请操作，提高性能
- 动态内存分配：对于中间临时的内存需求，可以进行临时申请和释放，节省内存使用，提高模型并发能力，但需要一定的开销
- 内存复用：对于同一类同一个大小的内存形式，又满足临时性，可以复用内存地址，减少内存申请。

根据 3.3.2 节实验过程中发现，在基于 MLIR 中间表示 IR 进行推理时，采用静态内存分配策略，需要一次分配所有张量的内存空间，对硬件内存提出了较高要求，本章针对此问题提出了内存分配的自动优化策略。

本章首先分析了当前 MLIR 系统中推理引擎的执行流程以及内存分配策略，针对其中存在的问题进行研究，结合当前策略和内存复用技术进行改进，最后通过实验验证该优化方法的有效性。

4.1 研究动机

深度学习框架如 TensorFlow、PyTorch 和 TPU-MLIR 对深度学习模型进行推理时往往基于算子库分发机制，通常情况下算子库针对特定硬件进行深度优化，难以修改，且大多数硬件厂商并不开源其算子库代码。通过对 TPU-MLIR 编译框架的推理引擎进行研究，在对 Yolov5s 模型和 Resnet50 模型进行 FP8 量化模拟时，基于 MLIR 中间表示的 CPU 推理引擎对于深度学习模型使用静态内存分配策略，一次性为模型中所有的算子的输入和输出分配全部的内存空间，之后调用对应的算子库进行模型推理。该方案的优点是推理速度快，对于较小的模

中国知网 https://www.cnki.net

型十分友好，但对于稍大一些的模型很可能导致内存不足。

当前面向深度学习的 CPU 内存优化的相关工作比较少，深度学习模型推理往往借助异构硬件进行加速推理，但一些模型验证场景仍需要 CPU 进行推理。当前深度学习框架普遍使用基于线程管理的 malloc 库，如 TensorFlow 使用的 ptmalloc[41]、jemalloc[42]、tcmalloc[43]，这些库通常基于线程进行内存管理，每个线程都有一个局部的内存池进行小内存分配和一个全局的内存进行大内存分配，但深度学习模型推理过程中，模型中张量通常是 MB 级别的大内存分配，运行时总量可达 GB 级别，这将需要从操作系统进行内存分配，导致大量的缺页中断带来性能的损失。

内存复用策略的求解是一个 NP 完全的问题。许多深度学习框架通常采用贪心的策略去分配内存，例如采用经典的动态内存分配 BestFit 算法，每次直接从内存池中选取可以满足条件的最小内存块，然而这种贪心的策略往往会陷入局部最优解，而无法求得全局最优解。为了更好地逼近内存分配策略全局最优解，MindSpore 框架提出了一种新的内存分配算法 SOMAS(Safe Optimized Memory Allocation Solver)[44]。SOMAS 针对异构环境将计算图并行流与数据依赖进行聚合分析，得到算子间祖先关系，构建张量全局生命周期互斥约束，使用多种启发式算法求解最优的内存规划，实现逼近理论极限的内存复用，从而提升支持的内存大小。黄奕桐[45]等提出了一种动态感知的深度学习内存分配器，在运行时收集内存分配信息并生成内存分配计划，然后通过模拟分配的方式动态调整生成的内存分配计划，这种动态内存分配方案在运行时带来了一定的开销，对于一些中小型模型，性能提升效果较差。Le T D、Zhang J 等人[46]主要研究在 CPU 和 GPU 异构计算架构上进行内存优化的策略，倾向节省 GPU 显存，缺乏对 CPU 内存使用的关注。

本节通过结合 MLIR 中间表示对网络模型分析，获取网络算子之间的数据流及控制流关系，基于固定 CPU 算子库，对 TPU-MLIR 模型推理引擎 CPU 内存占用进行改进，进一步减少模型执行过程中的内存占用峰值，为更大模型部署提供可能。

4.2 实现方法

本章首先研究了基于 MLIR 中间表示和算子库分发机制的 CPU 推理引擎设

中国知网 https://www.cnki.net

计，接着对当前系统中的静态内存分配策略进行分析，根据 MLIR 中间表示建模的深度学习模型特征，对模型的数据流进行梳理，对特定类型的算子进行内存复用优化。通过在运行时进行模型推理，对运行时操作系统内存峰值占用进行监测，评估内存优化的效果。

4.2.1 内存分配策略分析

TPU-MLIR 编译框架根据编译流程中不同阶段生成的 MLIR 中间表示设计了一套强大的推理引擎，根据图 3.4 的编译流程可知，其内部主要分为 TOP、TPU 和 bmodel 三个阶段，对应二种中间表示，其中 TOP 层和 TPU 层 IR 的推理基于 Intel oneDNN 算子库在 CPU 上进行执行，对模型编译结果进行快速验证，bmodel 层使用 flatbuffer 格式进行序列化存储，基于 TPU 算子库进行推理执行。本文主要关注 MLIR 中间表示的推理引擎优化，因此针对 TOP 层和 TPU 层的推理引擎进行研究，该部分推理引擎设计主要分为三个步骤：

（1）使用 MLIR 文件解析器对当前中间表示进行合法化解析获取模型表示的 Module 对象

（2）使用 MLIR 提供的 Walk 接口对 TOP Dialect 或 TPU Dialect 描述的深度学习模型进行遍历，遍历 Module 对象中的 Func 对象，针对 Func 对象中的 Op 算子进行内存分配，包括模型的输入 Input 算子和输出 Return 算子需要的内存、模型中各种算子的权重需要从权重文件中读取以填充到对应的缓冲区中。最后模型中各种算子的输出进行内存空间分配，空间分配完成后对权重文件进行释放以节省空间。

（3）使用 MLIR 提供的 Walk 接口对模型中的算子进行遍历，将步骤(2)中申请的内存空间以及权重缓冲区和对应算子的推理接口对象进行关联，即将推理接口对象中数据域指针指向之前申请的空间，该步骤形成了模型算子对应的输入和输出空间，将模型数据流串联起来，便于模型进一步展开推理。

（4）使用 invoke 模块对当前 MLIR 描述的 IR 文件按照数据流和控制流进行遍历，按照数据流对算子逐个调用对应的推理接口，即将缓冲区中的数据和权重作为算子库的输入进行计算，并将计算结果输出到对应的缓冲区中。当遇到 if 等控制流时，需要对条件语句进行推理计算，选择对应的分支按照数据流算子推理的方式进行推理执行，直至模型的 Return 算子，将最终输出写入 Return 算子对应的缓冲区中完成整个模型的推理。

中国知网 https://www.cnki.net

当前推理引擎的内存分配策略在模型调用 invoke 模块前便对所有的算子的输入和输出都申请了对应的内存空间，在模型执行期间不再需要申请和释放的额外内存空间，属于静态内存分配策略。优点是模型推理速度快，但要求设备的内存的最大容量必须大于步骤 2 分配的内存空间，否则会造成内存分配失败，推理引擎无法执行。

与静态内存分配相对的是动态内存分配，即在推理过程中根据算子的需要，即时的申请内存空间，由于深度学习模型中算子的输入或输出的内存空间往往以 MB 计算，即时申请需要较大的内存空间，会在一定程度上引发操作系统的缺页中断，进而降低模型的推理速度。通过对模型的 TOP 层和 TPU 层 MLIR 中间表示文件进行分析，发现模型中存在一些冗余的内存分配，可以通过内存复用策略减少静态内存分配和推理阶段的内存峰值占用。内存复用策略基于静态内存分配方案实现，在模型推理的内存分配阶段实现，本文在 4.2.2 节对该优化方案进行详细介绍。

4.2.2 内存复用策略支持

内存复用是通过多次对同一块内存单元进行读写的操作，其目的是提高内存的利用率，让有限的设备内存容纳更大的模型。通过 4.2.1 节分析，TPU-MLIR 框架在对 MLIR 中间表示描述的模型进行推理过程中，算子输出缓存区用于接收由算子库计算完成后的输出数据，经过后续算子读取后，当前内存缓冲区内数据往往不再被使用。此时可以将其他算子的计算结果写入该缓冲区进行原地内存复用。网络模型中各个算子的输入和输出之间存在一定的数据流和控制流关系，某个算子的输出可能作为一个或者多个后续算子的输入，因此在进行原地内存复用时需分析 MLIR 中间表示描述的计算图的特点，在不影响模型推理结果的前提下进行原地内存复用。

TOP 和 TPU Dialect 的设计在 MLIR 编译器中主要用于处理 Tensor 数据类型，处于编译器中较高的抽象层次，使用 TOP 和 TPU Dialect 描述的深度学习模型中算子的输入和输出对象都是一种基于 SSA(Static Single Assignment)的 Value 表示，MLIR 编译器为每个 Value 对象维护一个包含定义和使用关系的 D-U 链(Define Use Chain)[48]，可以通过 Value 对象快速获取其使用者集合。借助 MLIR 编译器提供的 D-U 链，可以高效的分析算子的输入和输出的使用情况，对满足依赖关系的输入或输出空间进行原地内存复用。

中国知网 https://www.cnki.net

在 3.3.2 节中，使用了 Yolov5s 模型对 FP8 类型进行了量化模拟工作，通过对模型的进一步观察发现，TOP 层和 TPU 层描述的 Yolov5s 模型具备大量 element-wise 逐元素操作算子，这些算子作为独立的网络层，在内存分配阶段同样需要为其分配对应的输入和输出空间，这些算子的输入和输出占据了整个网络中相当一部分内存空间，存在优化的空间。

原地内存复用还需要考虑内存空间的大小，根据推理引擎的设计，内存空间由算子的输出张量的形状和元素类型决定。当前 Yolov5s 网络和 Resnet50 网络中的逐元素算子如激活算子 SILU，其输入和输出具有相同的张量形状和类型，符合内存复用条件。通过修改推理引擎中的静态内存申请策略，通过对特定算子的输入缓冲区进行定义使用分析，获取缓冲区的全局使用数量，当使用数量为 1 即仅仅是当前算子使用时，为当前算子的输出和输入分配同一块内存空间，进行原地内存复用。以 SILU 激活算子为例整体思路如图 4.1 所示。

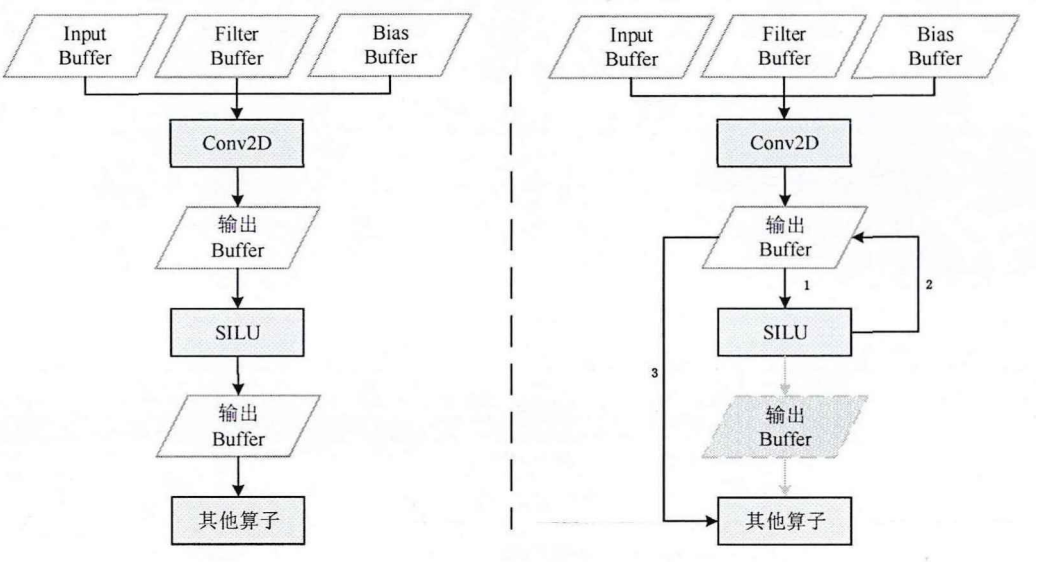


图 4.1 SILU 算子内存复用支持

图 4.1 展示了激活算子的内存复用策略，Add 等算子与之类似，同样需要判断输入的使用者数量。由于 Add 算子为二元操作，其输入可能具有不同的形状，在内存分配时应选择更大的输入形状对应的空间进行复用。

算法 4.1 描述了特定激活算子实现原地内存复用的流程，算法省去了对 MLIR 中间表示描述模型的遍历步骤，通过 isa 函数对模型中激活类算子进行定位，接

中国知网 https://www.cnki.net

着获取激活算子的输入 Value 对象，该 Value 对象作为内存原地复用的候选空间。为了进一步验证该空间的复用合法性，需要对 Value 对象的定义使用链进行分析，使用 distance 函数判断该空间的全局使用者数量，当使用者数量为 1 时，对激活算子的输出空间指针指向输入空间，否则为激活算子输出空间申请对应的内存缓冲区。对于 Add 等其他逐元素操作算子，需要加入额外的判断，选择符合条件的输入缓冲区进行原地内存复用。使用该优化方法前后其 MLIR 中间表示描述的模型没有任何区别，内存复用过程由 TPU-MLIR 编译框架在内存分配阶段自动完成。

算法 4.1：推理引擎算子内存复用策略

Input: element-wise 类算子如激活算子
Output: 进行内存复用分配后的激活算子

```
1 if isa<tpu::ActiveOp>(op) then
2    activeop = dyn_cast<tpu::ActiveOp>(op);
3    v = activeop.getInput();
4    /*获取算子输入内存空间的使用者数量*/
5    num= distance(v.user_begin(),v.user_end());
6    opname = getName(v);
7    /*param为算子输入参数，map容器存储所有算子输入和输出的内存空间对应
8                                                    的指针*/
9    param->inputs= map[opname]->data();
10    if num_uses == 1 then
11       /*将输出空间的指针指向输入空间地址*/
12       param->outputs =map[opname]->data();
13    else
14       result = activeop ->getResults();
15       o_name = getName(result);
16       param->outputs = map[o_name]->data();
17    end if
18 end if
```

4.3 实验结果与分析

本实验在 AMD R7 5800 CPU 上进行，其 TPU-MLIR 平台版本、深度学习模

中国知网 https://www.cnki.net

型架构及模型参数与 3.3.1 节实验环境相同。其中模型推理使用的算子库为 Intel oneAPI 深度神经网络库 oneDNN,oneDNN 是一个跨平台的 CPU 性能库，其中包含用于深度学习应用程序的基本构建块如 CNN 基元卷积和池化、RNN 原语和各种激活函数用于支持主流深度学习模型的推理。该库针对英特尔架构处理器、英特尔处理器显卡和 Xe 架构显卡进行了优化。支持 X86、Arm* 64 位架构(AArch64)和 OpenPOWER* Power ISA (PPC64)等其他架构。

本节实验以 TPU-MLIR 编译框架当前版本的系统测试性能为基准，对比进行原地内存复用算法优化后的系统内存峰值占用，同时考虑了算法对模型的可拓展性，选择了多种深度学习模型进行测试。为确保测试的合理性，每组数据进行了 20 次测试取得平均值，内存大小以 MB 为单位。

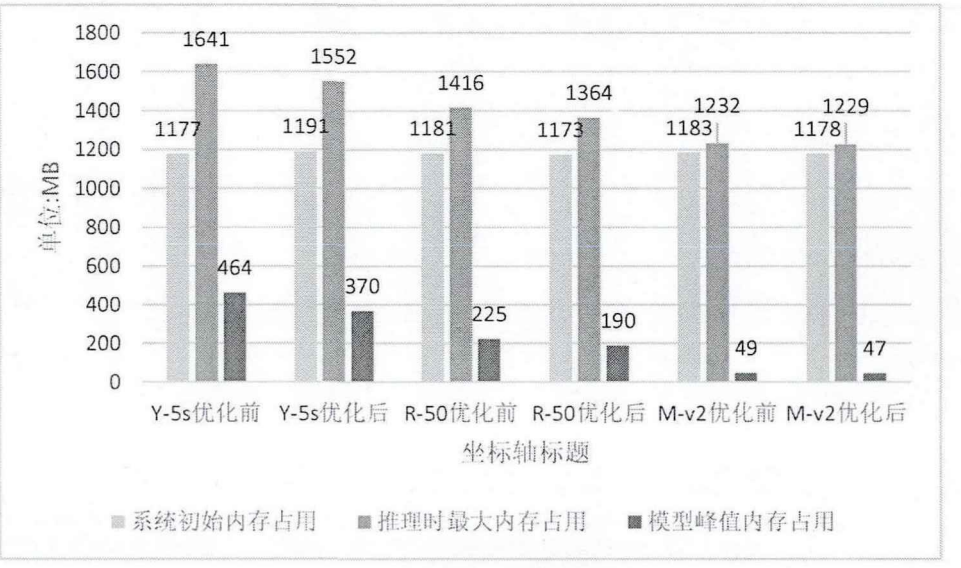


图 4.2 内存占用峰值对比

根据实验结果显示，如图 4.2 所示，3 种深度学习模型在经过优化前后的推理时间几乎完全一致。对于 Yolov5s 网络中依赖关系单一的 element-wise 算子，如 SILU 和 Add 算子，经过定义使用链分析，算子输入 Value 对象的使用者数量为 1，具备内存复用条件。通过多轮测试对比，内存复用前系统内存峰值占用约为 450MB~460MB 之间，复用后系统内存占用峰值在 366MB~375MB 之间，降低约 20%，效果提升明显。对于 Resnet50v1-7 网络，其中也存在一些依赖关系单一的 element-wise 算子，具备原地内存复用条件，数据如图 12 所示，优化前后内存峰值占用下降约 15%。在对 Mobilenetv2[49]的网络测试中，由于系统在模

中国知网 https://www.cnki.net

型接入时特定该网络实现了激活算子优化，因此该策略只对网络中的 add 算子进行了原地内存复用，内存峰值占用下降约 4%。

当前一些比较常见的网络模型如 ViT-Base、EfficientNet、VGG 等普遍存在大量的逐元素操作算子。对于具有如加法 AddOp、减法算子 SubOp，各种激活函数如 SigmoidOp、比较算子 CompOp、与或非算子等特征的网络，都可以借助该策略进行内存缓冲区使用情况分析，通过将算子的输入与输出复用同一块内存缓冲区，优化模型推理过程中的内存占用。在硬件方面，当前内存复用方案在 CPU 端验证可以达到较好的效果，进一步的在个人移动终端设备如 ARM 硬件或边缘计算芯片如 RSIC-V 硬件上进行推理，也可以借助提出的内存复用技术在模型部署前对模型的数据流进行分析，改善模型在推理阶段的内存分配策略，节省内存空间，为更大的模型部署提供可能。

4.4 本章总结

本章基于 TPU-MLIR 编译框架，对框架中的 MLIR 中间表示推理引擎开展研究，详细叙述了基于 MLIR 中间表示的推理引擎实现过程以及引擎的内存分配策略，接着通过对 MLIR 中间表示的定义使用链和深度学习模型的特征分析，在推理引擎的内存分配阶段提出了自动的原地内存复用策略，并对该策略的自动化的实现过程进行了详细的阐述。通过实验证明了该自动优化技术对深度学习模型推理时的内存峰值具有良好的改进效果，最后在章节的末尾分析了将该方法扩展到其他场景的可行性。

中国知网 https://www.cnki.net

5 基于 MLIR 的水平算子融合优化

深度学习模型的网络结构由算子和张量组成，算子性能的好坏直接决定了整个模型的执行效率。算子按其对资源的需求分为两类：访存密集型算子和计算密集型算子。计算密集型算子的执行时间主要耗费在计算上，如卷积、矩阵乘等；而访存密集型算子主要耗费在存储介质的访问上，例如 Elementwise、Broadcast 和 Reduce 类型算子。Elementwise 类型算子特征是可以在不改变张量形状的前提下，对张量进行逐元素计算。这类算子对输入张量中的每个元素进行相同的操作，输出张量的形状与输入张量的形状相同。Broadcast 类型算子对不同形状的张量进行逐元素计算，主要包括加法、减法、乘法、除法等。Broadcast 类型算子通常会对较小的张量进行扩展，以使得在逐元素计算时具有相同的形状。Reduce 类型算子通常是对张量沿着指定的维度进行规约操作。该过程会使输入张量在规约维度上压缩，而在其他维度上保持不变。

在基于 MLIR 的编译器中，算子融合往往基于处理张量数据类型的 High-Level Dialect 进行，通过编译器内部 Pass 指导对 MLIR 中间表示描述的深度学习模型根据规则进行 IR 变换，以实现多个算子融合为单个高效内核，进而提升模型推理效率，但当前 MLIR 编译器的融合规则缺少对水平算子融合的支持。本章针对大语言模型中常见的 SwiGLU 算子[50]展开算子融合研究，通过分析算子的计算模式与算子生成策略，在 MLIR 编译器中自动算子融合的基础上，开展自动水平算子融合优化，通过进一步在算子生成环节优化算子内部计算模式，提升算子执行效率。

5.1 研究动机

5.1.1 问题研究

在本文的第 5 章开展基于 MLIR 的推理引擎内存优化时发现，TPU-MLIR 编译框架中的模型高效推理依赖特定硬件的算子库支持，如基于 CPU 的 oneDNN 和基于 TPU 的 TPU 算子库。针对一些新算子和融合算子的支持不仅仅需要对 MLIR 中间表示描述的深度学习模型进行 Pass 优化，更需要根据算子特征修改算子库细节，这对程序性能优化人员提出了较高的要求。当前模型越来越庞大，

中国知网 https://www.cnki.net

算子种类越来越丰富，对模型进行算子融合优化以及融合后的算子生成仍是通过编译器自动提升模型推理性能的主要方案。

当前的语言大模型 Meta llama2[51]和阿里 Qwen[52]都基于 Transformer Decoder 架构[53]，如图 5.1 所示，其中主要包含 Attention 算子和 FFN 算子，对于 Attention 算子的加速研究，FlashAttention[54]中提出了多种加速策略，其中存在大量的算子融合研究，但针对 FFN 层特别是新提出的 SwiGLU 算子，其优化仍存在一定的提升空间。

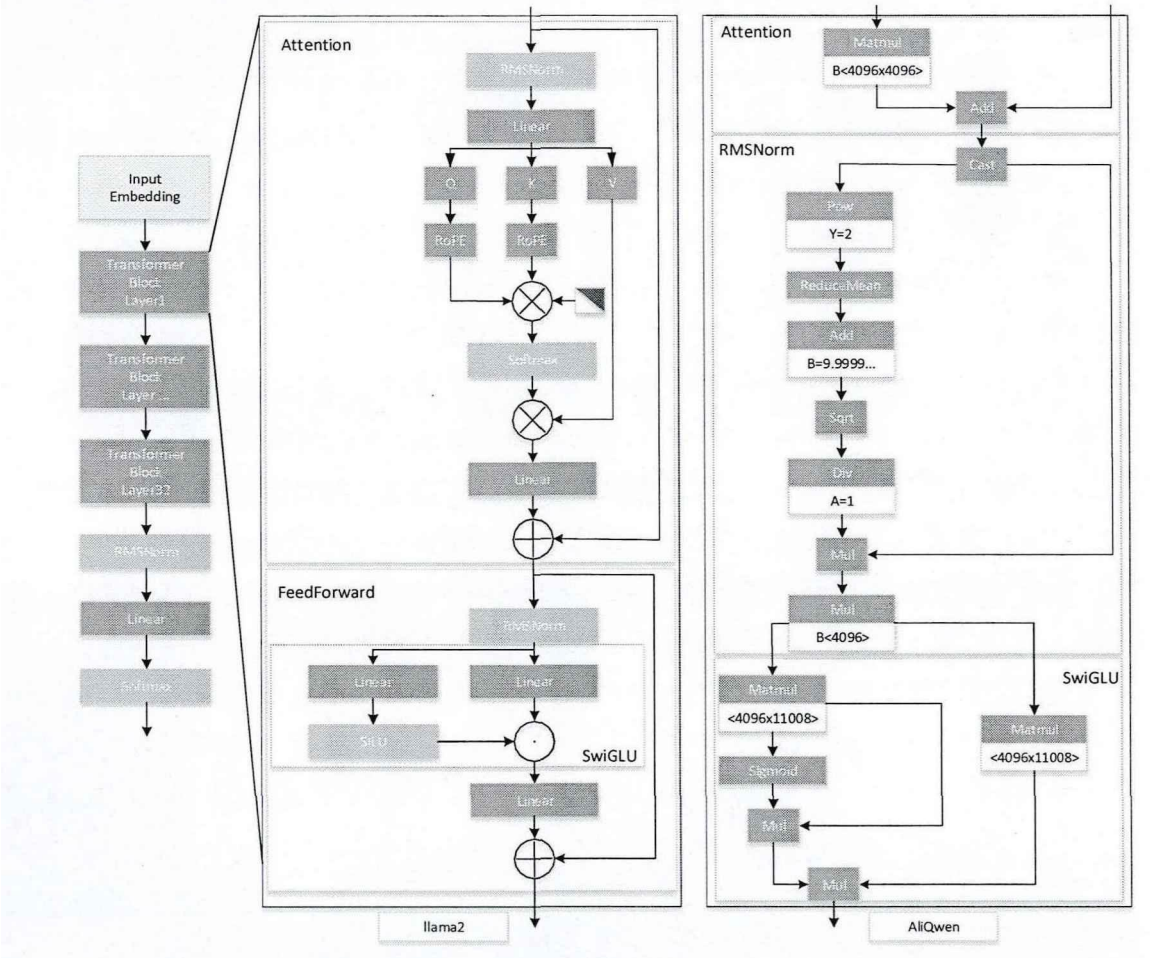


图 5.1 语言模型中的 SwiGLU 算子

通过对 FFN 层中 SwiGLU 算子的计算模式研究，如图 5.1 中阿里 Qwen FFN 算子的计算细节，发现该算子由多个计算步骤，对应多个不同的算子表示，存在算子融合的机会。通过将 SwiGLU 算子接入 MLIR 编译器中发现 MLIR 编译

中国知网 https://www.cnki.net

器对水平位置的算子融合缺少支持。本章基于 MLIR 编译器对 SwiGLU 算子进行水平融合研究，同时借助 MLIR 编译器的自动算子生成技术进一步探索融合后算子的优化空间。

5.1.2 相关工作

在 1.2 节中对当前深度学习编译器的算子融合技术研究进行了概述，这些研究主要针对垂直位置的算子，水平算子融合方面的研究工作较少，Ao Li 等人[55]提出了一种 GPU 内核的自动水平融合策略，通过对 Batchnorm、Im2Col、MaxPool、Upsample 算子进行配对融合，通过在内核中加入控制流对线程进行划分提高 GPU 并行度以改善总体程序性能，但仅测试了两个算子的水平融合，未能结合常见的垂直算子融合同时也缺少对 SwiGLU 算子的融合研究。在 Meta 的推理引擎 AI_Template[56]中提供了垂直融合、水平融合和内存融合三大类融合规则，其中水平融合可以把处于平行的 GEMM、LayerNorm 以及其他不同输入 shape 的算子融合为一个 GPU 算子。Google 深度学习编译器 XLA[57]中通过 Pass 支持将处于水平位置的算子融合为一个大的算子，可以把不同形状的 kernel 水平的融合在一起减少内核启动的开销从而提升性能。区别与传统的垂直算子融合模式，通过减少算子间中间数据的冗余访存，提升程序性能，水平融合更多的是利用减少 GPU 内核启动次数，通过内核融合提供深度融合的机会，充分利用 GPU 资源以提升模型性能。

5.2 实现方法

本章以 SwiGLU 算子为例，先对 SwiGLU 算子的计算模式进行初步介绍，通过对 MLIR 编译器中间表示建模的 SwiGLU 算子分析，基于 MLIR 编译器开展算子的水平融合研究。接着介绍基于 MLIR 编译器的自动算子生成策略对融合算子进行深度优化的实现过程，针对合并后的算子在算子生成过程存在的冗余指令，通过指令删除进一步优化算子在硬件上的执行效率。

5.2.1 SwiGLU 算子计算模式分析

SwiGLU 是语言大模型中常见的算子属于 GLU[58]的变体，其计算公式如 5.1 所示。

$$SwiGLU(x,W,V,b,c) = Swish\beta(xW+b)\otimes(xV+c) \tag{5.1}$$

中国知网 https://www.cnki.net

其中的 Swish 函数[59]是对带有非零负值梯度的 ReLU 平滑版本，计算公式如 5.2 所示。

$$Swish_{\beta}(x) = x \otimes \sigma(\beta x) \tag{5.2}$$

其中参数β默认为 1，σ表示 sigmoid 函数，⊗为逐元素乘，x 为输入参数，W 为权重参数，实验中不包含 bias 项。简化后的计算公式如 5.3 所示。

$$SwiGLU(x, W, V) = (xW) \otimes \sigma(xW) \otimes (xV) \tag{5.3}$$

该算子在 MLIR 编译器内主要由 Linalg Dialect 线性代数模块[60]进行表示，在 Linalg 层中根据算子计算步骤，SwiGLU 算子由多个 Linalg 算子组成。Linalg Dialect 中主要分为具名算子（namedOp）和常规算子（genericOp）,具名 Op 主要用于表示常见的线性代数算子比如矩阵乘法算子、矩阵向量乘算子、Attention 算子等，而一些非常见的代数计算如 Elementwise、Broadcast 和 Reduce 类型算子则由 Linalg 常规算子表达。从结构上看，Linalg 算子主要用于表达完美嵌套循环，每个 Linalg 算子主要分为循环结构域和负载域，如图 5.2 中，结构域中包含 index_map 用于表达每个循环维度上索引的计算方式，利用多面体表达[61]方式更好的进行访存；iterator_type 用于表示每个循环维度的分布方式，主要分为可并行 parallel 和规约 reduce 两种；ins、outs 用于描述输入和输出；负载域由算子附带的 region 表示由花括号标识，用于携带循环内的计算。这种结构化 IR 设计[62]有助于对循环结构和计算负载进行分离，对线性代数计算进行结构化表达，使得 MLIR 中优化遍 Pass 的设计更加通用。



图 5.2 MLIR 中间表示描述的 SwiGLU 算子

当前 SwiGLU 算子的输入会根据不同使用场景发生改变，主要对应 Linalg 具名算子中矩阵乘法和矩阵向量乘算子，其他计算则由 Linalg 常规算子进行描

中国知网 https://www.cnki.net

述，Linalg 描述的 SwiGLU 算子的 MLIR 中间表示如图 5.2 所示，其中 module 表示 SwiGLU 算子整体，forward 函数表示算子的执行数据流。可以看到，SwiGLU 算子根据计算过程被分为 5 个 Linalg 算子进行描述，其首先根据输入和权重参数进行第一次矩阵乘法或矩阵向量乘，接着将产生的结果作为 Linalg 常规算子描述的 sigmoid 算子的输入进行计算，之后将 sigmoid 算子的计算结果和矩阵乘法算子的计算结果进行逐元素相乘，这便完成了 Swish 算子的计算。最后将输入和输入的第二个权重参数进行相乘，并将计算结果和 Swish 算子的结果进行逐元素乘法获取最终 SwiGLU 算子的输出。整体计算过程由图 5.2 所示。

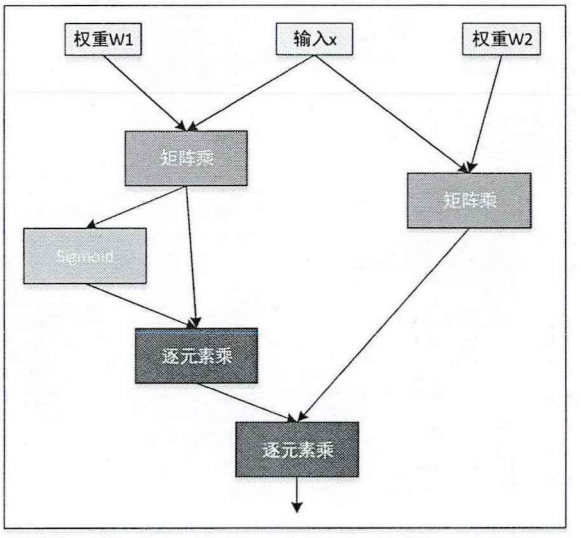


图 5.3 SwiGLU 计算模式

5.2.2 水平融合策略支持

通过对SwiGLU算子的计算模式以及SwiGLU算子的MLIR中间表示分析，SwiGLU 算子中主要包含计算密集型算子如矩阵乘法算子和访存密集型算子如 sigmoid 激活函数、逐元素乘法和矩阵向量乘算子，存在算子融合的机会，即将计算密集型算子矩阵乘法和后续的访存密集型算子如sigmoid激活函数进行垂直融合，MLIR 编译器也提供了基础的算子融合优化，主要分为以下几个步骤：

（1）对 SwiGLU 算子的 MLIR 中间表示进行合法性校验获取算子的 Module 对象。

（2）根据 MLIR 的中间表示设计，从 Module 到 Func 对象到 Region 对象到 Block 对象到 Op 对象，查找具有 reduce 轴的 Linalg 算子，并将该算子标记为

中国知网 https://www.cnki.net

rootOp。

（3）处理每个 rootOp，首先针对 rootOp 的每个输入获取输入 Value 对象的生产者算子，通过 Value 对象的 D-U 链分析以及对算子生产者算子的支配树分析确定当前生产者是否能与 rootOp 融合为一个算子，如果融合成功则融合后算子作为 rootOp 继续分析算子的输入 Value 和生产者算子继续融合直到融合失败。接着针对当前融合 op 的输出 Value 进行分析，获取 Value 的消费者算子，根据分析消费者算子的支配关系和 Value 对象的 D-U 链确定当前 rootOp 是否能跟后续消费者算子进行融合，如果融合成功，则融合后算子作为 rootOp 继续向消费者融合直至融合完成。

（4）经过步骤 3 的分析，符合融合规则的算子组，在 IR 层面使用一个 dispatch region 即分发域进行标识，表示在后续算子生成和推理时以分发域为单位生成和启动内核。

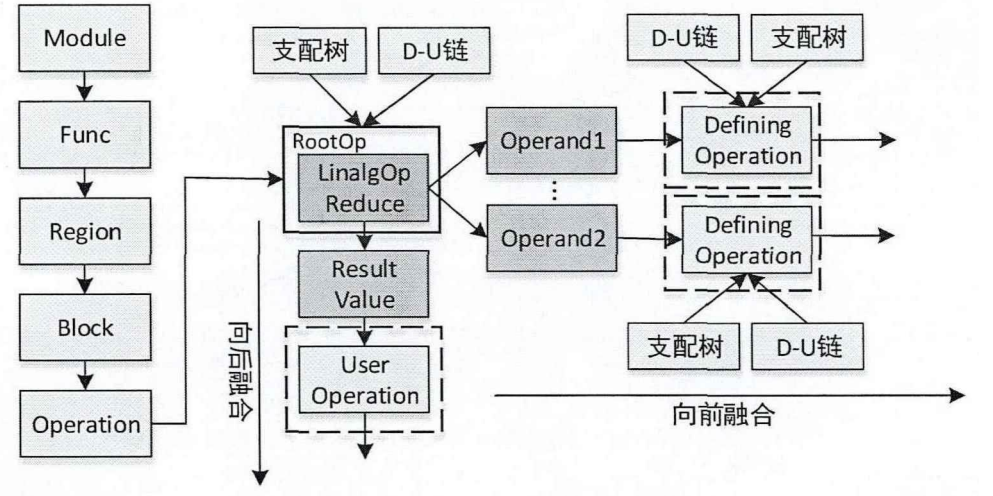


图 5.4 MLIR 算子融合规则

对于 SwiGLU 算子，经过 MLIR 编译器进行自动算子融合后发现一共生成了 2 个内核，第一个内核用于计算输入和第二个权重的矩阵乘法，之后将计算结果作为第二个内核的输入启动第二个内核推理，第二个内核中矩阵乘法作为带有规约轴的 rootOp，不断向其输出的消费者融合，与 sigmoid、逐元素乘法算子合并为一个完整的算子，图 5.5 中描述了当前 SwiGLU 算子的融合情况。

中国知网 https://www.cnki.net

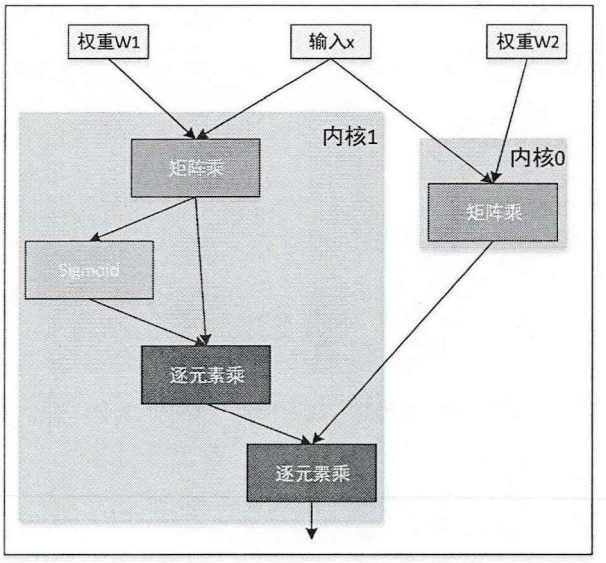


图 5.5 MLIR 自动算子融合后的 SwiGLU 算子

通过图 5.5 可以观察到，内核 2 中算子与内核 1 中矩阵乘法算子几乎处于水平位置，该算子只有 2 个输入，且有 1 个输入与内核 1 中矩阵乘法算子相同，只有逐元素乘这一个算子作为该算子的消费者，其使用者仅仅为 1。同时该算子计算量较小，却需要单独开启一次内核，因此该算子适合融合到内核 1 中，这样不仅可以减少内核的启动，同时对于融合后的算子具备更多的算子内优化空间。对于 SwiGLU 算子的水平融合，需要在不影响原来垂直融合的基础上，进一步的进行水平融合，通过对当前 MLIR 算子融合算法的研究发现，若算法的输入中间表示中已经存在了融合完成的分发域，不会处理域内的任何算子。因此，考虑在 MLIR 编译器中添加一条水平融合优化遍，对 SwiGLU 的 MLIR 中间表示进行优化后，再进行其他部分的算子融合。该 Pass 优化遍整体如算法 5.1 所示。

与 MLIR 中算子融合策略类似，该算法首先接收校验合法的 SwiGLU 算子的 MLIR 中间表示，基于 Linalg Dialect 描述的中间表示按照 MLIR 的设计由 Module 到 Func 逐层匹配直到识别出逐元素乘法算子，接着需要处理该算子的输入操作数的生产者，判断生产者算子是否依次为逐元素乘算子和矩阵乘法算子，接着需要继续判断生产者算子的输入对应的生产者算子直至符合 SwiGLU 算子的计算模式，除此之外，还需要注意算子间的数据流是否匹配，是否存在其他

中国知网 https://www.cnki.net

分支数据流，最终将符合的算子序列加入到容器中，使用 MLIR 提供的分发域接口，将融合内的所有算子加入到同一个分发域中，以便于再后续算子生成环节作为单独的内核进行算子生成和推理。

算法 5.1： SwiGLU 算子水平融合优化

```
Input:  SwiGLU 算子的 MLIR 中间表示代码
Output: 融合后的 SwiGLU 算子的 MLIR 中间表示代码
1 if (isGenericMultiplyOp(genericOp).failed())  return failure();
2 /*匹配并校验当前算子的类型是否为乘法算子,采用倒置顺序匹配*/
3 genericMultiplyOp =
4                 genericOp->getOperand(0).getDefiningOp<linalg::GenericOp>();
5 matmulOp1 =
6       genericOp->getOperand(1).getDefiningOp<linalg::MatmulOp>();
7 if (!genericMultiplyOp || !matmulOp1 ||
8       isGenericMultiplyOp(genericMultiplyOp).failed())  return failure();
9 /*匹配并校验当前乘法算子的输入对应的算子类型*/
10 genericSigmoidOp =
11        genericMultiplyOp->getOperand(0).getDefiningOp<linalg::GenericOp>();
12 matmulOp2 = genericMultiplyOp->getOperand(1)
13                       .getDefiningOp<linalg::MatmulOp>();
14 if (!genericSigmoidOp || !matmulOp2 ||
15        isGenericSigmoidOp(genericSigmoidOp).failed())  return failure();
16 /*继续匹配对应的 sigmoid 算子和矩阵乘法算子*/
17 genericSigmoidOpInput =genericSigmoidOp->getOperand(0)
18                                       .getDefiningOp<linalg::MatmulOp>();
19 if (!genericSigmoidOpInput || genericSigmoidOpInput != matmulOp2)
20       return failure();
21 /*对 sigmoid 输入数据流进行匹配，要求输入的定义算子为矩阵乘法算子*/
22 matmulOp1Input = matmulOp1->getOperand(0);
23 matmulOp2Input = matmulOp2->getOperand(0);
24 if (!matmulOp1Input || !matmulOp2Input || matmulOp1Input != matmulOp2Input)
25       return failure();
26 SmallVector<Operation *> opsToFuse = {
27      matmulOp2, genericSigmoidOp, genericMultiplyOp, matmulOp1, genericOp};
28 /*将待融合的算子都加入到容器中*/
29 for (Operation *op : opsToFuse) {
30       if (!IREE::Flow::isNonNullAndOutsideDispatch(op))  return failure();}
31 result = wrapConsecutiveOpsInDispatchRegion(rewriter, opsToFuse);
32 /*将容器中算子打包进一个内核,如果算子已经融合到其他内核则融合失败*/
```


中国知网 https://www.cnki.net

经过算法 5.1，SwiGLU 算子同时将垂直位置的算子和水平位置的算子融合到同一个内核中，形成的内核如图 5.6 所示。

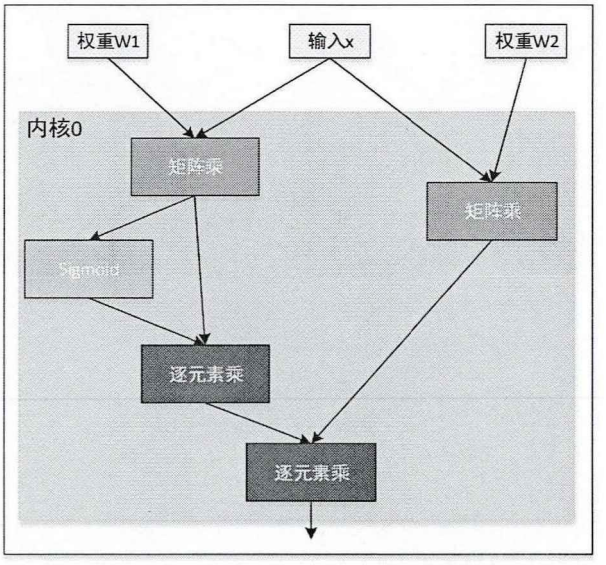


图 5.6 水平加垂直融合后的 SwiGLU 算子

通过对 SwiGLU 算子的 CUDA 代码生成策略研究发现，MLIR 编译器首先将 SwiGLU 算子拆分主机侧 CPU 端和设备侧 GPU 端，主机侧主要负责模型输入数据的初始化和模型中内核函数调用，维护模型中的数据流和控制流，对于设备端主要是针对每个内核进行单独的 GPU 代码生成和优化。

MLIR 自动算子生成主要基于 CUDA 的 SIMT 模型，主要针对矩阵乘法算子，基于编译器内部设定的分块大小配置和输入矩阵的 shape 进行矩阵切分即将整个矩阵计算分配到多个 Block 间进行并行执行，之后针对每个 Block 内部的矩阵乘法，进一步的划分到以线程为单位进行并行执行，后续还会进行共享内存申请，数据拷贝并行化、延迟隐藏、bank 冲突消除等优化，其中针对算子融合的主要是在 SwiGLU 算子递降到 Vector Dialect 描述的阶段，针对内存的读写进行优化，该部分通过访存的读写消除算法消除了垂直算子间不必要的读写，提升了程序的性能。通过对该部分优化后的中间表示研究发现，针对矩阵乘法的优化，会在任何情况下将计算的结果写回内存，同时进行计算结果的全局线程同步 barrier，针对 SwiGLU 算子中的矩阵乘法，通过 D-U 链分析，没有其他的使用者时，可以进行共享内存写回的消除，同时，针对该算子中两个矩阵乘法计

中国知网 https://www.cnki.net

算后的同步，由于同步会使得当前 block 内线程全部等待，而第一个矩阵乘法计算后的不影响第二个矩阵乘法的计算，这里存在的不必要的等待，可能造成性能的降低，通过定位该 barrier 的位置进行删除，进一步提升融合后算子的整体性能如图 5.7 和图 5.8 所示。

```
%47 = vector.transfer_read %alloc_6[%7, %c0], %cst_1 {in_bounds = [true, true]} : memref<32x36xf32, #gpu.address_space<workgroup>>, vector<4x32xf32>
%48 = vector.transfer_read %alloc_3[%c0, %8], %cst_1 {in_bounds = [true, true]} : memref<32x132xf32, #gpu.address_space<workgroup>>, vector<32x4xf32>
%49 = vector.contract {indexing_maps = [affine_map<(d0, d1, d2) -> (d0, d2)>, affine_map<(d0, d1, d2) -> (d2, d1)>, affine_map<(d0, d1, d2) -> (d0, d1)>], iterator_types = ["parallel",
gpu.barrier
scf.yield %49 : vector<4x4xf32>
}
vector.transfer_write %23, %alloc_5[%7, %8] {in_bounds = [true, true]} : vector<4x4xf32>, memref<32x132xf32, #gpu.address_space<workgroup>>
...
%47 = vector.transfer_read %alloc_2[%7, %c0], %cst_1 {in_bounds = [true, true]} : memref<32x36xf32, #gpu.address_space<workgroup>>, vector<4x32xf32>
%48 = vector.transfer_read %alloc_3[%c0, %8], %cst_1 {in_bounds = [true, true]} : memref<32x132xf32, #gpu.address_space<workgroup>>, vector<32x4xf32>
%49 = vector.contract {indexing_maps = [affine_map<(d0, d1, d2) -> (d0, d2)>, affine_map<(d0, d1, d2) -> (d2, d1)>, affine_map<(d0, d1, d2) -> (d0, d1)>], iterator_types = ["parallel",
gpu.barrier
scf.yield %49 : vector<4x4xf32>
}
vector.transfer_write %28, %alloc_6[%7, %8] {in_bounds = [true, true]} : vector<4x4xf32>, memref<32x132xf32, #gpu.address_space<workgroup>>
%29 = arith.negf %23 : vector<4x4xf32>
...
```

图 5.7　存在访存冗余的 SwiGLU 算子

优化后在中间表示：

```
%47 = vector.transfer_read %alloc_4[%7, %c0], %cst_1 {in_bounds = [true, true]} : memref<32x36xf32, #gpu.address_space<workgroup>>, vector<4x32xf32>
%48 = vector.transfer_read %alloc_3[%c0, %8], %cst_1 {in_bounds = [true, true]} : memref<32x132xf32, #gpu.address_space<workgroup>>, vector<32x4xf32>
%49 = vector.contract {indexing_maps = [affine_map<(d0, d1, d2) -> (d0, d2)>, affine_map<(d0, d1, d2) -> (d2, d1)>, affine_map<(d0, d1, d2) -> (d0, d1)>], iterator_types = ["parallel",

scf.yield %49 : vector<4x4xf32>
}

...
%47 = vector.transfer_read %alloc_2[%7, %c0], %cst_1 {in_bounds = [true, true]} : memref<32x36xf32, #gpu.address_space<workgroup>>, vector<4x32xf32>
%48 = vector.transfer_read %alloc[%c0, %8], %cst_1 {in_bounds = [true, true]} : memref<32x132xf32, #gpu.address_space<workgroup>>, vector<32x4xf32>
%49 = vector.contract {indexing_maps = [affine_map<(d0, d1, d2) -> (d0, d2)>, affine_map<(d0, d1, d2) -> (d2, d1)>, affine_map<(d0, d1, d2) -> (d0, d1)>], iterator_types = ["parallel",
gpu.barrier
scf.yield %49 : vector<4x4xf32>
}

%29 = arith.negf %23 : vector<4x4xf32>
...
```

图 5.8　访存冗余消除后的 SwiGLU 算子

5.3　实验结果与分析

本实验选择在 PyTorch 中搭建 SwiGLU 算子，通过 Torch-MLIR 项目[63]将包含该算子的深度学习模型由 Python 描述转换为 MLIR 中间表示描述，然后由基于 MLIR 构建的 IREE 编译器进行算子融合优化与 CUDA 算子生成。

基于 MLIR 编译基础设施构建的 IREE 编译器如图 5.9 所示，不同于 TPU-MLIR 编译框架，IREE 编译器采用自动算子生成策略，复用了 MLIR 中较多的内置 Dialect，如 Func、Arith、Lianlg、Vector、GPU、NVVM、Affine、SCF、MemRef 等，除此之外还构建了特定的 Flow Dialect 用于开展高效计算图优化，Stream Dialect 用于建模模型调度，HAL Dialect 用于建模硬件抽象，以递降到不同的硬件后端。本文实验主要基于 IREE 编译器的 Flow Dialect 开展，并在 Nvidia GPU 上进行算子生成工作对融合算子进一步优化。

中国知网　https://www.cnki.net

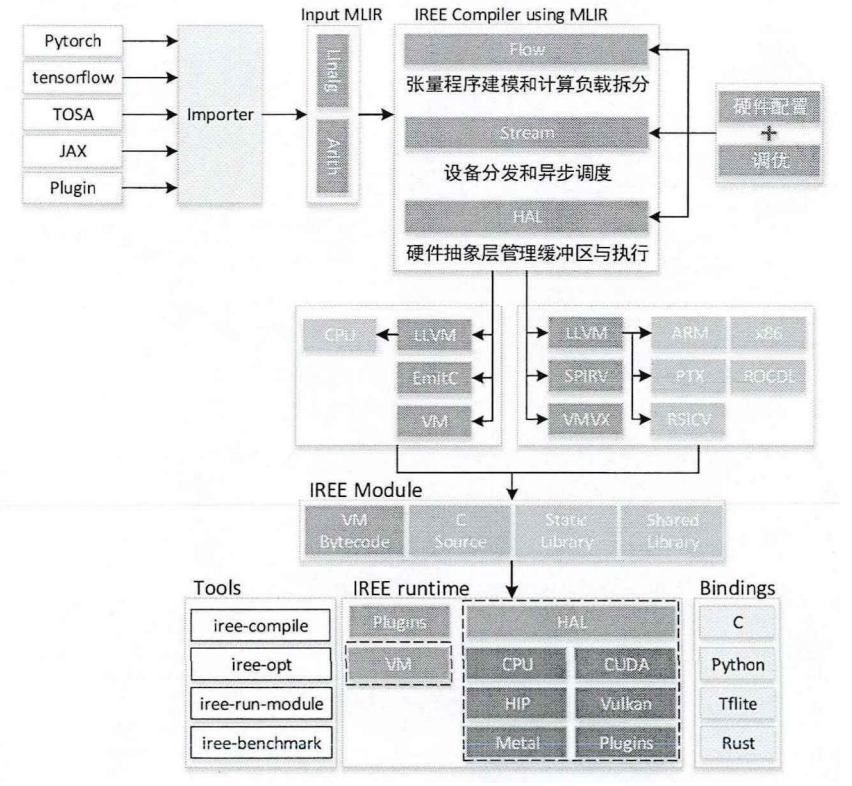


图 5.9 基于 MLIR 的 IREE 编译器架构

本实验在 Nvidia GPU 上进行实验，其 CPU 参数为 AMD R7 5800,GPU 为 RTX3060laptop，CUDA 版本为 12.2，Pytorch 版本为 2.3.0.dev20240101， IREE 版本为 735a77e4116cb91e2ad1d3a709ec5cf456b35c30，Torch-MLIR 版本为 20240104.1073。实验以当前版本的 IREE 编译器为基准，使用 Google benchmark 测试工具对包含 SwiGLU 算子的模型进行推理测试，对比进行水平算子融合和访存冗余优化后的算子推理时间，以毫秒为单位。

通过输入不同的输入 shape，包含 SwiGLU 算子的模型在不同矩阵乘法下使用联合优化技术与优化前的推理时间对比如图 5.10 所示。从图 5.10 可以直观的看出，当输入张量大小为 1x5120 时加速比最大，程序加速性能最好推理时间提升约 9%，这是因为当第一个维度为 1 时，矩阵乘法算子转换为矩阵向量乘算子，此时由计算密集型算子转换为访存密集型算子，对算子进行融合会提升 GPU 硬件资源的使用率，使得计算和访存更加平衡，而当 shape 不断增大时，GPU 计算资源占用不断增加，对 GPU 硬件提出了更高的要求，通过算子融合优化后 SwiGLU 算子在多种输入下，总体都有性能的提升。

中国知网 https://www.cnki.net

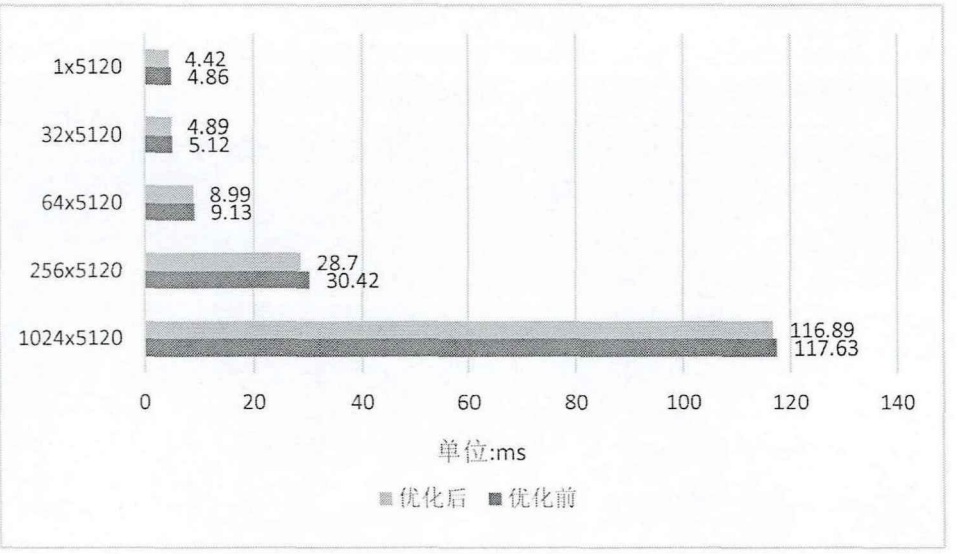


图 5.10 不同输入下的优化前后推理时间对比

由于自回归语言模型仅在第一次推理时根据输入进行矩阵乘法算子调用，接着通过调用矩阵向量乘算子产生后续输出，因此本实验对语言模型推理速度在一定程度上具有优化作用，当前优化策略可以扩展到其他包含 SwiGLU 算子的深度学校模型中享受改优化带来的性能提升，同时针对深度学校模型中其他位于水平位置的算子也可以采取相似的融合策略进行优化，进一步提升模型的整体推理速度。

5.4 本章总结

本章实现了一种基于 MLIR 编译器 Linalg Dialect 层的算子融合策略优化方法，对 SwiGLU 算子进行算子融合优化。通过添加编译器优化遍的方式对输入的 SwiGLU 算子进行模式匹配，采用倒序匹配的方式逐步的将算子融合到一个内核中，该策略在支持算子垂直融合的基础上进行了水平融合支持，其次根据 MLIR 编译器对融合后算子的算子生成策略研究，对算子中存在的冗余访存指令和 barrier 指令进行删除，进一步改善了算子的效率。最后通过实验对模型算子的推理速度进行比较，证实了本章算子融合策略优化方法的有效性。

中国知网 https://www.cnki.net

6 总结与展望

6.1 工作总结

随着深度学习模型编译优化的快速发展，利用 MLIR 编译器基础设施开展编译优化得到了越来越多的关注和运用，如何充分挖掘模型推理过程的优化空间已经成为目前研究的重要方向。本文基于 MLIR 编译器，进行面向 CPU 平台的低精度量化模拟、推理引擎优化和面向异构平台的算子融合优化研究，致力于使用 MLIR 编译器提升深度学习模型的推理性能。

本文的主要贡献为：

（1）介绍了 MLIR 编译器的设计和深度学习模型量化技术，探索了在 MLIR 编译器中开展 FP8 数据类型量化对多种深度学习模型推理精度的影响。通过分析深度学习模型在 MLIR 编译器中的量化流程，在编译器中构建了 FP32 和 FP8 数据类型间的转换算法，为深度学习模型在 MLIR 编译器中开展 FP8 量化模拟提供支持。

（2）研究了基于 MLIR 的推理引擎内存分配策略，通过对深度学习模型特征以及模型的 MLIR 中间表示代码结构研究，发现模型在当前推理引擎下的静态内存分配策略存在优化空间，根据模型中算子的定义使用关系分析，最终实现了一套基于 MLIR 编译器的自动内存原地复用优化策略，改善了深度学习模型推理的内存占用。

（3）探索了算子融合和算子生成技术，通过 SwiGLU 算子的计算模型分析，研究该算子在 MLIR 编译器中的自动算子融合和算子生成策略，提出了一种结合垂直算子融合和水平算子融合的优化策略，同时结合算子的 GPU 代码生成流程，提出了一种访存优化方法，改善了 SwiGLU 算子在异构设备上的推理性能。

本文基于 MLIR 编译器针对深度学习模型开展的三项研究，不仅仅注重 MLIR 编译器的前端和中端的优化，还对编译器后端的算子生成环节展开了深入研究，其中对于 FP8 数据类型的量化模拟作为一种优化策略，不受模型架构和算子库影响，适用于其他模型的 FP8 量化精度评估；内存原地复用方案在 CPU 端验证可以达到较好的效果，进一步可以在个人移动终端设备如 ARM 硬件或边缘计算芯片如 RSIC-V 硬件上进行推理，通过在模型部署前对模型的数据流和控

中国知网 https://www.cnki.net

制流分析，改善模型在推理阶段的内存占用。最后，对于提出的算子融合策略，同样适用于具有水平位置算子特征的其他模型或算子，具备广泛的扩展性。

6.2 未来展望

经过以上内容的研究，本文使用 MLIR 编译器开展深度学习模型编译优化研究在一些模型和算子上得到了性能的提升。针对深度学习模型的 FP8 量化模拟研究，对 Yolov5s 和 Resnet50 模型的精度损失进行了系统的评估，同时在精度验证期间针对基于 MLIR 的推理引擎展开了内存原地复用优化，节省了模型验证所需的内存空间，然而 FP8 数据类型量化研究和推理引擎内存优化依然有很大的优化空间，如提供更多类型的深度学习模型特别是大语言模型的验证，探索更多的内存复用机会等，因此在本文的研究基础上后续还可以进行以下优化：

（1）在针对深度学习模型开展 FP8 量化模拟研究时，仅仅针对在 CPU 后端上进行模型的精度验证，使用了混合精度测试，为了进一步探索 FP8 量化推理的效果，考虑在比如 Nvidia Hopper 架构的 GPU 上开展 FP8 数据类型的模型推理工作，对于深度学习模型的类别，可以开展对大语言模型的 FP8 数据类型量化，探索不同数值对模型推理的效果，选取合适的数制进行模型的推理。

（2）第四章节描述的仅仅是针对深度学习模型静态内存分配策略下的原地内存复用方案，对于更大的语言模型，可能需要开展动态内存的分配方案，这需要在模型的推理期间即时的进行内存的申请和释放，可以通过对深度学习模型的中间表示数据流和控制流分析，探索并规划在推理期间动态内存复用机会，减少因内存申请释放带来的性能开销，提升模型的推理性能。

（3）第五章节描述的优化方案仅仅是针对 SwiGLU 算子的融合和内核生成优化。前面分析了 SwiGLU 算子的 MLIR 中间表示代码结构，可以考虑将其他模型中类似处于水平位置的算子进行融合，以进一步提升模型的效率，同时因为当前大语言模型对硬件设备提出了较高的要求，开展大语言模型的编译优化需要同时借助强大的算力和低精度量化类型的硬件支持，因硬件限制本文未能将融合后的算子进行整体模型的测试，可以考虑使用 MLIR 编译器对大语言模型展开更为全面的编译优化，提升大模型在硬件的部署效率，最后还可以进一步的讲算子编译性能与主流深度学习框架和算子库进行对比以供参考。

中国知网 https://www.cnki.net

参考文献

[1] Abadi M. TensorFlow: learning functions at scale[C]. Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming. 2016, 1-1.

[2] Paszke A, Gross S, Massa F, et al. Pytorch: An imperative style, high-performance deep learning library[J]. Advances in neural information processing systems, 2019.

[3] Google. "machine learning compiler ecosystem" [EB]. 2023. https://github.com/openxla/xla.

[4] Chen T, Moreau T, Jiang Z, et al. TVM: An automated end-to-end optimizing compiler for deep learning[C]. Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation. 2018, 579-594.

[5] Lattner C, Amini M, Bondhugula U, et al. MLIR: Scaling compiler infrastructure for domain specific computation[C]. 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). IEEE, 2021, 2-14.

[6] Lattner C, Adve V. LLVM: A compilation framework for lifelong program analysis & transformation[C]. International symposium on Code Generation and Optimization(CGO). IEEE, 2004, 75-86.

[7] Pengchao Hu, Man Lu, Lei Wang, et al. TPU-MLIR: A Compiler For TPU Using MLIR[J]. CoRR abs/2210.15016 (2022).

[8] Hsin-I Cindy Liu, Marius Brehler, Mahesh Ravishankar, et al. TinyIREE: An ML Execution Environment for Embedded Systems From Compilation to Deployment[J]. IEEE Micro 42(5),2022, 9-16.

[9] Zhen Zheng, Zaifeng Pan, et al. BladeDISC: Optimizing Dynamic Shape Machine Learning Workloads via Compiler Approach[J]. Proc. ACM Manag. Data 1(3):2023,206:1-206:29.

[10] Hongbin Zhang. "基于 MLIR 搭建领域特定语言与领域特定架构间桥梁" [EB]. 2022. https://github.com/buddy-compiler/buddy-mlir.

[11] ByteDance. "基于 MLIR 构建深度学习模型在多种硬件后端的编译解决方案" [EB]. 2023. https://github.com/bytedance/byteir.

[12] Google. " A retargetable MLIR-based machine learning compiler and runtime toolkit." [EB]. 2021. https://github.com/openxla/iree.

[13] OpenAI. " Development repository for the Triton language and compiler." [EB]. 2021. https://github.com/openai/triton.

[14] Scott Cyphers, Arjun K. Bansal, Anahita Bhiwandiwalla, et al.Intel nGraph: An Intermediate Representation, Compiler, and Executor for Deep Learning[J]. CoRR abs/1801.08058 (2018).

[15] SiFive. " Deploy and Optimize ML E2E models on RISC-V platform via OpenXLA." [EB]. 2023. https://www.youtube.com/@openxla/playlists.

[16] Benoit Jacob, Skirmantas Kligys, Bo Chen, et al. Quantization and Training of Neural

中国知网 https://www.cnki.net

Networks for Efficient Integer-Arithmetic-Only Inference[C]. CVPR 2018: 2704-2713.

[17] Xiao Sun, Jungwook Choi, Chia-Yu Chen, et al. Hybrid 8-bit Floating Point (HFP8) Training and Inference for Deep Neural Networks[C]. NeurIPS 2019: 4901-4910.

[18] Paulius Micikevicius, Dusan Stosic, Neil Burgess, et al. FP8 Formats for Deep Learning[J]. CoRR abs/2209.05433 (2022)

[19] Chris Leary and Todd Wang. XLA: TensorFlow, compiled[C]. TensorFlow Dev Summit (2017).

[20] Ma L, Xie Z, Yang Z, et al. Rammer: Enabling holistic deep learning compiler optimizations with rtasks[C]. Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation. 2020, 881-897.

[21] Niu W, Guan J, Wang Y, et al. DNNFusion: accelerating deep neural networks execution with advanced operator fusion[C]. Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021, 883-898.

[22] Zheng Z, Yang X, Zhao P, et al. AStitch: enabling a new multi-dimensional optimization space for memory-intensive ML training and inference on modern SIMT architectures[C]. Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2022, 359-373.

[23] Zhu H, Wu R, Diao Y, et al. {ROLLER}: Fast and Efficient Tensor Compilation for Deep Learning[C]. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). 2022, 233-248.

[24] Jia Z, Padon O, Thomas J, et al. TASO: optimizing deep learning computation with automatic generation of graph substitutions[C]. Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019, 47-62.

[25] Zhao J, Gao X, Xia R, et al. Apollo: Automatic partition-based operator fusion through layer by layer optimization[J]. Proceedings of Machine Learning and Systems, 2022, 4: 1-19.

[26] Zhao J, Li B, Nie W, et al. AKG: automatic kernel generation for neural processing units using polyhedral transformations[C]. Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021, 1233-1248.

[27] Mahesh Ravishankar, Hanhan Wang, Nicolas Vasilache. Fusion of Elementwise operations using Linalg on tensors[C]. MLIR Open Design Meeting, 2020.

[28] MLIR. "MLIR核心Dialect" [EB]. 2019. https://mlir.llvm.org/docs/Dialects.

[29] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper[J]. CoRR abs/1806.08342 (2018).

[30] Badreddine Noune, Philip Jones, Daniel Justus, et al. 8-bit Numerical Formats for Deep Neural Networks[J]. CoRR abs/2206.02915 (2022).

[31] Andrey Kuzmin, Mart van Baalen, Yuwei Ren, et al. FP8 Quantization: The Power of the Exponent[C].NeurIPS 2022.

[32] IEEE. Std 754-2019 (Revision of IEEE 754-2008) IEEE Standard for Floating-Point Arithmetic[S]. doi: 10.1109/IEEESTD.2019.8766229, 2019.

中国知网 https://www.cnki.net

[33] Dhiraj D. Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, et al. A Study of BFLOAT16 for Deep Learning Training[J]. CoRR abs/1905.12322 (2019).

[34] John Cheng，Max Grossman，Ty McKercher. CUDA C编程权威指南[M]，颜成钢，殷建，李亮等译. 北京：机械工业出版社，2017. 338~408

[35] Gaskill B. Onnx: the open neural network exchange format[J]. Linux J, 2018.

[36] Jianhui Li, Zhennan Qin,Yijie Mei, et al. oneDNN Graph Compiler: A Hybrid Approach for High-Performance Deep Learning Compilation [J]. Co-RR abs/2301.01333 (2023)

[37] Xudong Zhou, Ye Meng, Xin Xinxin, et al. Research of YOLOv5s Model Acceleration Strategy in AI Chip[C].ICCCS 2023: 791-794.

[38] Tsung-Yi Lin, Michael Maire, Serge J. Belongie, et al. Microsoft COCO: Common Objects in Context[C].ECCV (5) 2014: 740-755.

[39] Kaiming He, Xiangyu Zhang, Shaoqing Ren, et al. Deep Residual Learning for Image Recognition[C].CVPR 2016: 770-778.

[40] Jia Deng, Wei Dong, Richard Socher, et al. ImageNet: A large-scale hierarchical image database[C].CVPR 2009: 248-255.

[41] GLOGER W. Dynamic memory allocator implementations in linux system libraries[EB]. 2006. http://www.malloc.de/en/index.html.

[42] EVANS J. A scalable concurrent malloc(3)implementation for FreeBSD[C].Proceedings of the BSDCan Conference.2006.

[43] GHEMAWAT S,MENAGE P. TCMalloc:Thread-caching malloc[EB]. 2007. https:// goog-perftools.sourceforge.net/doc/tcmalloc.html.

[44] Huawei. "A new open source deep learning training/inference framework that could be used for mobile, edge and cloud scenarios." [EB]. 2019. https://www.mindspore.cn/.

[45] 黄奕桐. 面向深度学习的计算调度和内存优化[D].[硕士学位论文].安徽:中国科学技术大学,2023.

[46] Zhang J , Yeung S H , Shu Y ,et al. Efficient Memory Management for GPU-based Deep Learning Systems[J]. 2019.DOI:10.48550/arXiv.1903.06631.

[47] Le T D , Imai H , Negishi Y ,et al. Automatic GPU memory management for large neural models in TensorFlow[C]. Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management. 2019.DOI:10.1145/3315573.3329984.

[48] R. Allen and K. Kennedy. Optimizing Compilers for Modern Architectures: A Dependence-based Approach[M], Massachusetts:Morgan Kaufmann, 2001.

[49] Mark Sandler, Andrew G. Howard, Menglong Zhu, et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks[C]. CVPR 2018: 4510-4520.

[50] Noam Shazeer. GLU Variants Improve Transformer[J]. CoRR abs/2002.05202 (2020).

[51] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models[J]. CoRR abs/2307.09288 (2023).

中国知网 https://www.cnki.net

[52] Jinze Bai, Shuai Bai, Yunfei Chu, et al. Qwen Technical Report[J]. CoRR abs/2309.16609 (2023).

[53] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017.

[54] Tri Dao.FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning[J]. CoRR abs/2307.08691 (2023).

[55] Ao Li, Bojian Zheng, Gennady Pekhimenko, et al. Automatic Horizontal Fusion for GPU Kernels[C]. CGO 2022: 14-27.

[56] Meta. "A Python framework which renders neural network into high performance CUDA/HIP C++ code." [EB]. 2023. https://github.com/facebookincubator/AITemplate.

[57] Daniel Snider, Ruofan Liang. Operator Fusion in XLA: Analysis and Evaluation[J]. CoRR abs/2301.13062 (2023).

[58] Yann N. Dauphin, Angela Fan, Michael Auli, et al. Language Modeling with Gated Convolutional Networks[C]. ICML 2017: 933-941

[59] Ramachandran, Prajit et al. Swish: a Self-Gated Activation Function[J]. arXiv: Neural and Evolutionary Computing (2017).

[60] MLIR. "MLIR Linalg Dialect" [EB]. 2019. https://mlir.llvm.org/docs/Dialects/Linalg.

[61] 赵捷，李颖颖，赵荣彩.基于多面体模型的编译“黑魔法”[J].软件学报，2018，29(08):2371-2396.

[62] MLIR. " The case for a simplified polyhedral form" [EB]. 2019. https://mlir.llvm.org/docs/Rationale/RationaleSimplifiedPolyhedralForm.

[63] Google. "Aims to provide first class support from the PyTorch ecosystem to the MLIR ecosystem." [EB]. 2023. https://github.com/llvm/torch-mlir.

中国知网 https://www.cnki.net